

CS360 Introduction to Database

Query Optimization

Min-Soo Kim

School of Computing, KAIST

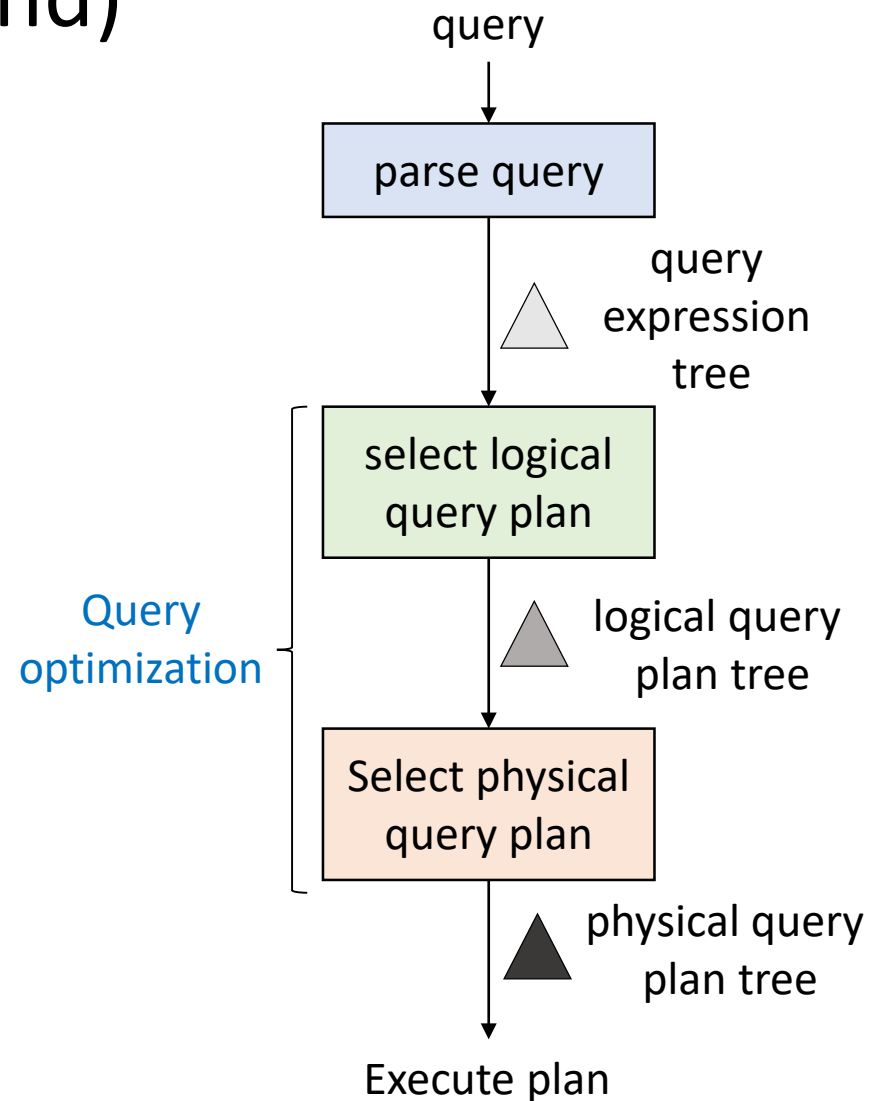
Outline

- Parsing
- Algebraic laws
- Cost Estimation
- Join order
- Cost-based Plan Selection

Query Compilation

Query compilation (remind)

1. **Parsing:** constructs a **parse tree** for the query
2. **Logical plan generation:** converts the parse tree to a **logical query plan**
 - query rewrite: transforms an initial plan into a more efficient equivalent plan
3. **Physical plan generation:** turn the logical plan into a **physical query plan**
 - selecting algorithms and an order



Syntax analysis and parse trees

- **Parsing**: convert a text written in a query language to a parse tree
- Nodes in a tree
 - **atoms**: lexical elements (e.g., SELECT), names of attributes or relations, constants, parentheses, operators (e.g., +, <), ...
 - leaf nodes in a tree (i.e., no children)
 - **syntactic categories**: names for families of query subparts
 - e.g., <Query>: represent some queries in the common select-from-where form
 - e.g., <Condition>: any expression following WHERE in SQL

Example grammar for a **simple** (not real) SQL

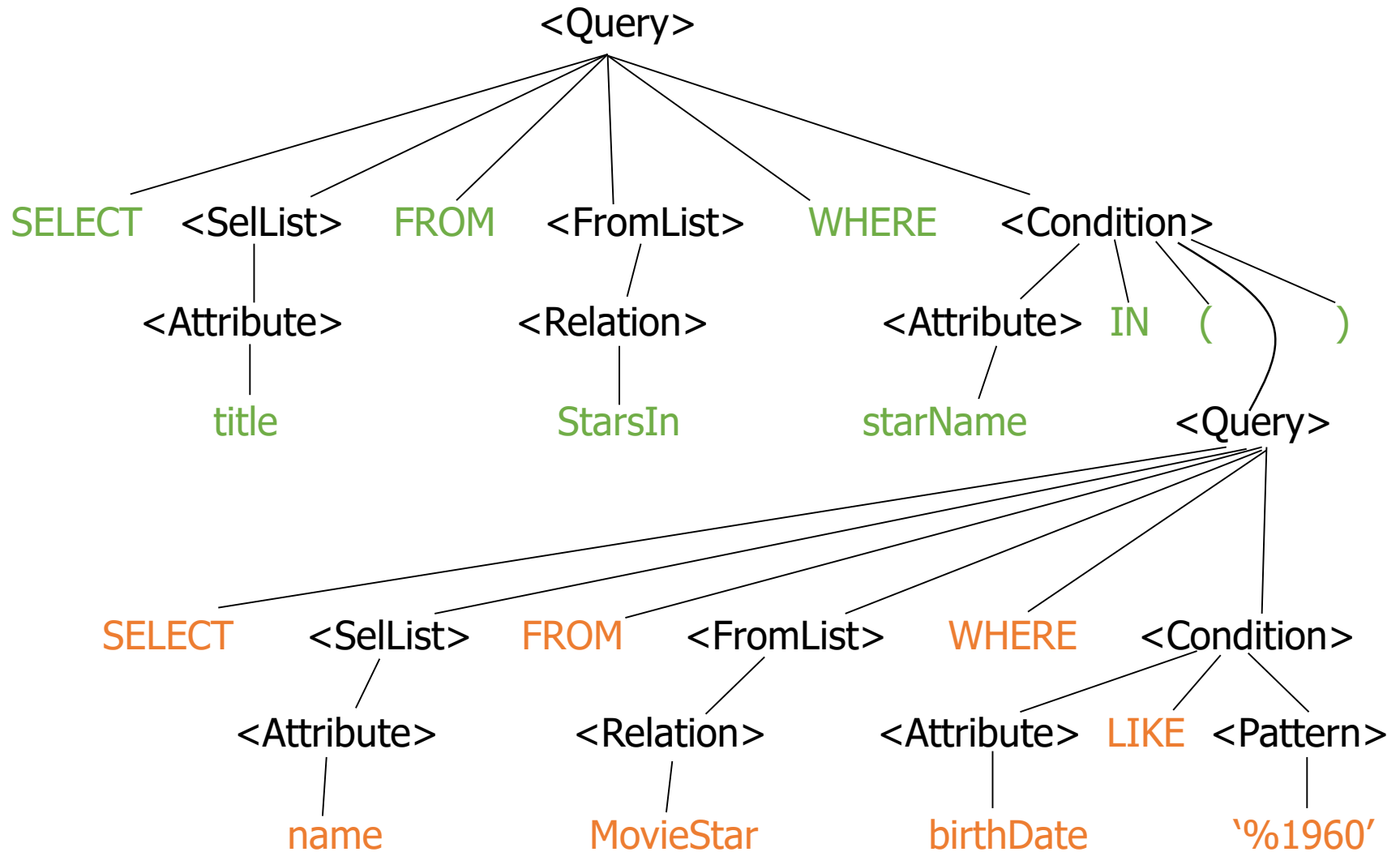
- $\langle \text{Query} \rangle ::= \text{SELECT } \langle \text{SelList} \rangle \text{ FROM } \langle \text{FromList} \rangle \text{ WHERE } \langle \text{Condition} \rangle$
- $\langle \text{SelList} \rangle ::= \langle \text{Attributes} \rangle, \langle \text{SelList} \rangle$
- $\langle \text{SelList} \rangle ::= \langle \text{Attributes} \rangle$
- $\langle \text{FromList} \rangle ::= \langle \text{Relation} \rangle, \langle \text{FromList} \rangle$
- $\langle \text{FromList} \rangle ::= \langle \text{Relation} \rangle$
- $\langle \text{Condition} \rangle ::= \langle \text{Condition} \rangle \text{ AND } \langle \text{Condition} \rangle$
- $\langle \text{Condition} \rangle ::= \langle \text{Attribute} \rangle \text{ IN } (\langle \text{Query} \rangle)$
- $\langle \text{Condition} \rangle ::= \langle \text{Attribute} \rangle = \langle \text{Attribute} \rangle$
- $\langle \text{Condition} \rangle ::= \langle \text{Attribute} \rangle \text{ LIKE } \langle \text{Pattern} \rangle$

Example: SQL query

```
SELECT title
FROM StarsIn
WHERE starName IN (
    SELECT name
    FROM MovieStar
    WHERE birthdate LIKE '%1960'
);
```

(Find the movies with stars born in 1960)

Example: Parse Tree

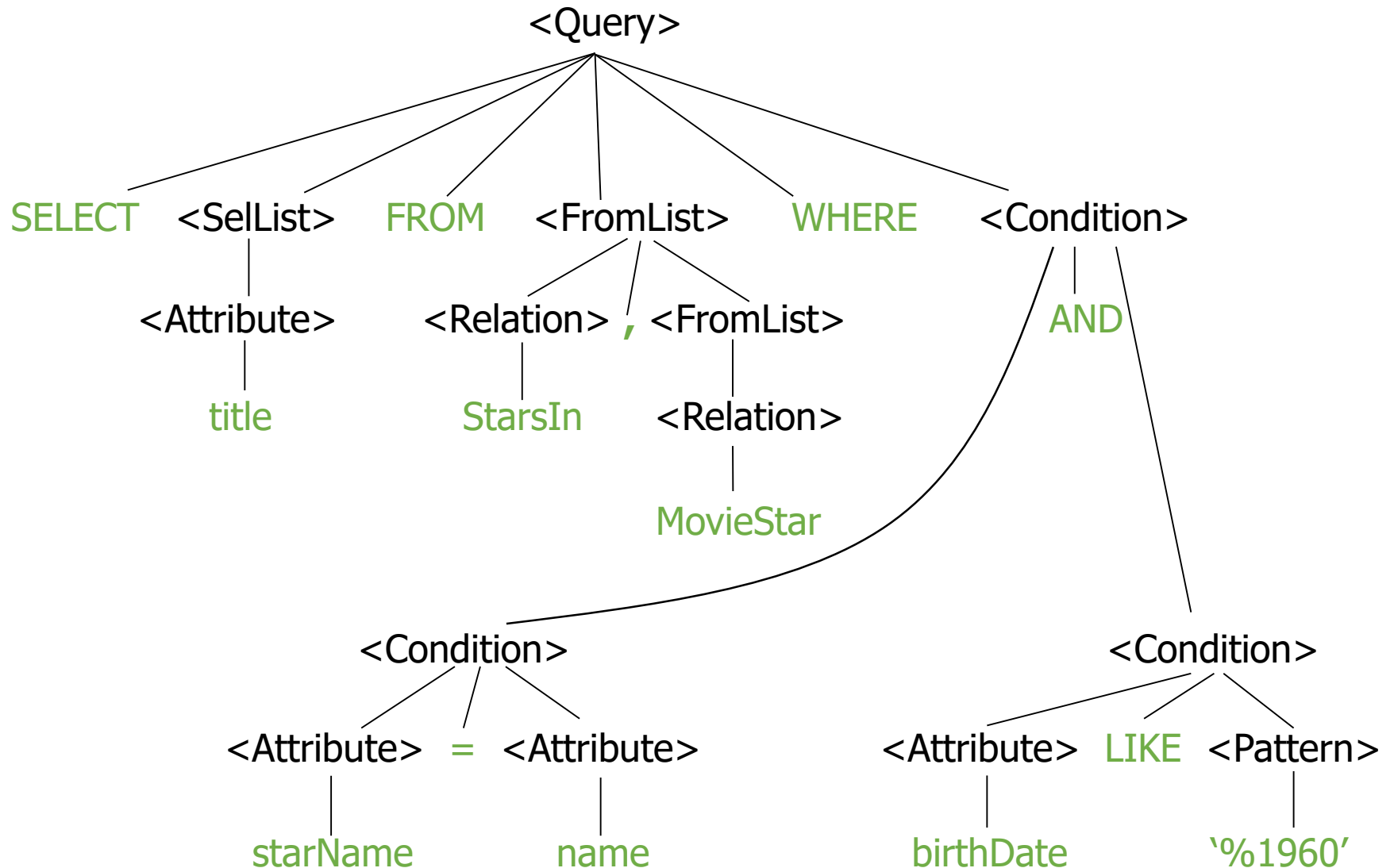


Example: SQL query (alternative)

```
SELECT title
FROM StarsIn, MovieStar
WHERE starName = name AND
      F.K.      P.K.
      birthdate LIKE '%1960';
```

(Find the movies with stars born in 1960)

Example: Parse Tree (alternative)



Preprocessing

- Semantic checking

- even if a query is valid syntactically, it may violate one or more semantic rules on the use of names

- ① Check relation uses

- every relation in FROM must be a relation or view

- ② Check and resolve attribute uses

- every attribute in SELECT/WHERE must be an attribute of some relation in the current scope
- resolve each attribute (e.g., StarsIn.title)

- ③ Check types

- birthdate must be a string or a compatible type for LIKE

Algebraic laws for improving query plans

- Algebraic laws
 - turn one expression tree into an equivalent expression tree
 - may have a more efficient physical query plan
- Query-rewrite phase
 - applying the algebraic transformations
 - result becomes the logical query plan

Commutative and associative laws

- The below laws hold for product ($\times$) and union ($\cup$)
 - laws for union hold for both sets and bags

$$R \bowtie S = S \bowtie R$$

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

- Associative law for theta-join cannot be applied arbitrarily
 - e.g., $R(a,b)$, $S(b,c)$, and $T(c,d)$

$$(R \bowtie_{R.b > S.b} S) \bowtie_{a < d} T$$

$$R \bowtie_{R.b > S.b} (S \bowtie_{a < d} T)$$

Distributive law of intersection over union

- Distributive laws hold for sets

$$A \cap_S (B \cup_S C) = (A \cap_S B) \cup_S (A \cap_S C)$$

- But, distributive laws do NOT hold for bags
 - e.g., $A = B = C = \{x\}$

$$A \cap_B (B \cup_B C) = \{x\} \cap_B \{x, x\} = \{x\}$$

$$(A \cap_B B) \cup_B (A \cap_B C) = \{x\} \cup_B \{x\} = \{x, x\}$$

Laws involving selection

- **Pushing selection**: a major tool for query optimization
- **Splitting laws**
 - break the condition into constituent parts if it is complex
 - the law for OR works only for sets

$$\sigma_{c_1 \text{ AND } c_2}(R) = \sigma_{c_1}(\sigma_{c_2} R) = \sigma_{c_2}(\sigma_{c_1} R)$$

$$\sigma_{c_1 \text{ OR } c_2}(R) = (\sigma_{c_1}(R)) \cup_s (\sigma_{c_2}(R))$$

- **Union:** must be pushed to both

$$\sigma_{c1}(R \cup S) = \sigma_c(R) \cup \sigma_c(S)$$

- **Difference:** must be pushed to the first argument
 - optionally may be pushed to the second

$$\sigma_{c1}(R - S) = \sigma_c(R) - S = \sigma_c(R) - \sigma_c(S)$$

- $\times, \cap, \bowtie, \bowtie_D$

- only push to a relation that has all the attributes mentioned in C
- e.g., $R(a,b)$ and $S(b,c)$

$$\begin{aligned} & \sigma_{(a=1 \text{ OR } a=3) \text{ AND } b < c}(R \bowtie S) \\ & \sigma_{a=1 \text{ OR } a=3}(\sigma_{b < c}(R \bowtie S)) \\ & \sigma_{a=1 \text{ OR } a=3}(R \bowtie \sigma_{b < c}(S)) \\ & \sigma_{a=1 \text{ OR } a=3}(R) \bowtie \sigma_{b < c}(S) \end{aligned}$$

by splitting law

by pushing $\sigma_{b < c}$

by pushing $\sigma_{a=1 \text{ OR } a=3}$

Laws involving projection

- Pushing down
 - quite usual for project to remain where it is after pushing down
 - less useful than pushing down selection
- Term $\pi_{E \rightarrow x}$
 - E: input attributes, x: output attribute
 - e.g., $\pi_{a+b \rightarrow x, c}(R)$
- e.g., $R(a, b, c)$ and $S(c, d, e)$

$$\begin{aligned} & \pi_{a+e \rightarrow x, b \rightarrow y}(R \bowtie S) \\ & \pi_{a+e \rightarrow x, b \rightarrow y}(\pi_{a,b,c}(R) \bowtie \pi_{c,e}(S)) \\ & \pi_{a+e \rightarrow x, b \rightarrow y}(R \bowtie \pi_{c,e}(S)) \end{aligned}$$

Laws about joins and products

- Applying the below rules from right to left

$$R \bowtie_{\text{c}} S = \sigma_{\text{c}}(R \times S)$$

$$R \bowtie S = \pi_{\text{L}}(\sigma_{\text{c}}(R \times S))$$

- **C**: condition that equates each pair of attributes from R and S with the same name
- **L**: list that includes one attribute from each equated pair and all the other attributes of R and S (e.g., {a, b, c, d, e} in prev. slide)
- Computing joins are generally much faster than computing a product followed by a selection and a projection

Laws involving duplicate elimination (δ)

- Moving δ down reduces the size of intermediate relations

$$\left\{ \begin{array}{l} \delta (R \times S) = \delta (R) \times \delta (S) \\ \delta (R \bowtie S) = \delta (R) \bowtie \delta (S) \\ \delta (R \bowtie_c S) = \delta (R) \bowtie_c \delta (S) \\ \delta (\sigma_c(R)) = \sigma_c(\delta(R)) \end{array} \right.$$

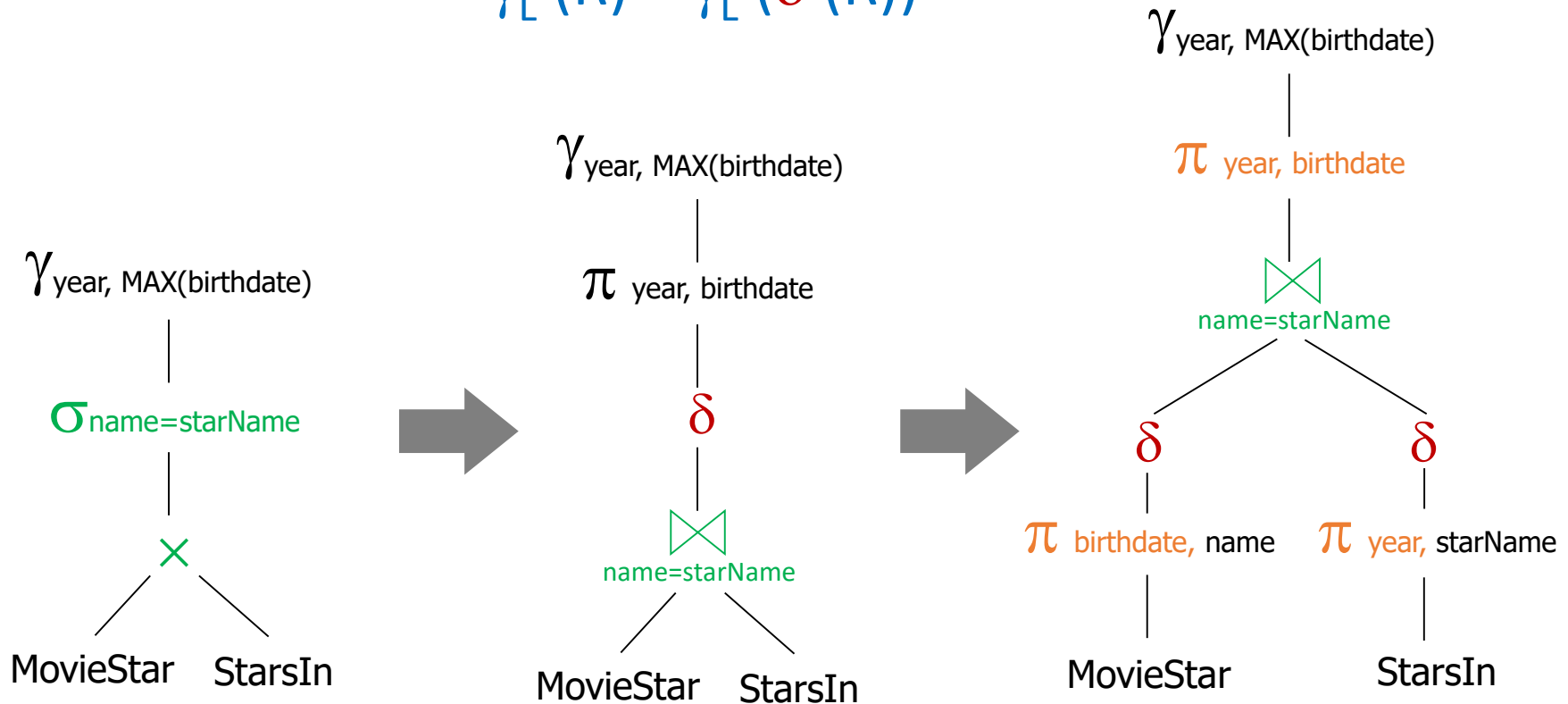
$$\delta (R \cap_B S) = \delta (R) \cap_B S = R \cap_B \delta (S) = \delta (R) \cap_B \delta (S)$$

- Cannot be pushed through $\cup_B$, $-_B$, or π
 - can be pushed through many, but not all operators

Laws involving grouping(γ) and aggregation

- SUM, COUNT, and AVG are not duplicate-impervious
- MIN/MAX: duplicate-impervious
 - not affected by the presence or absence of duplicates

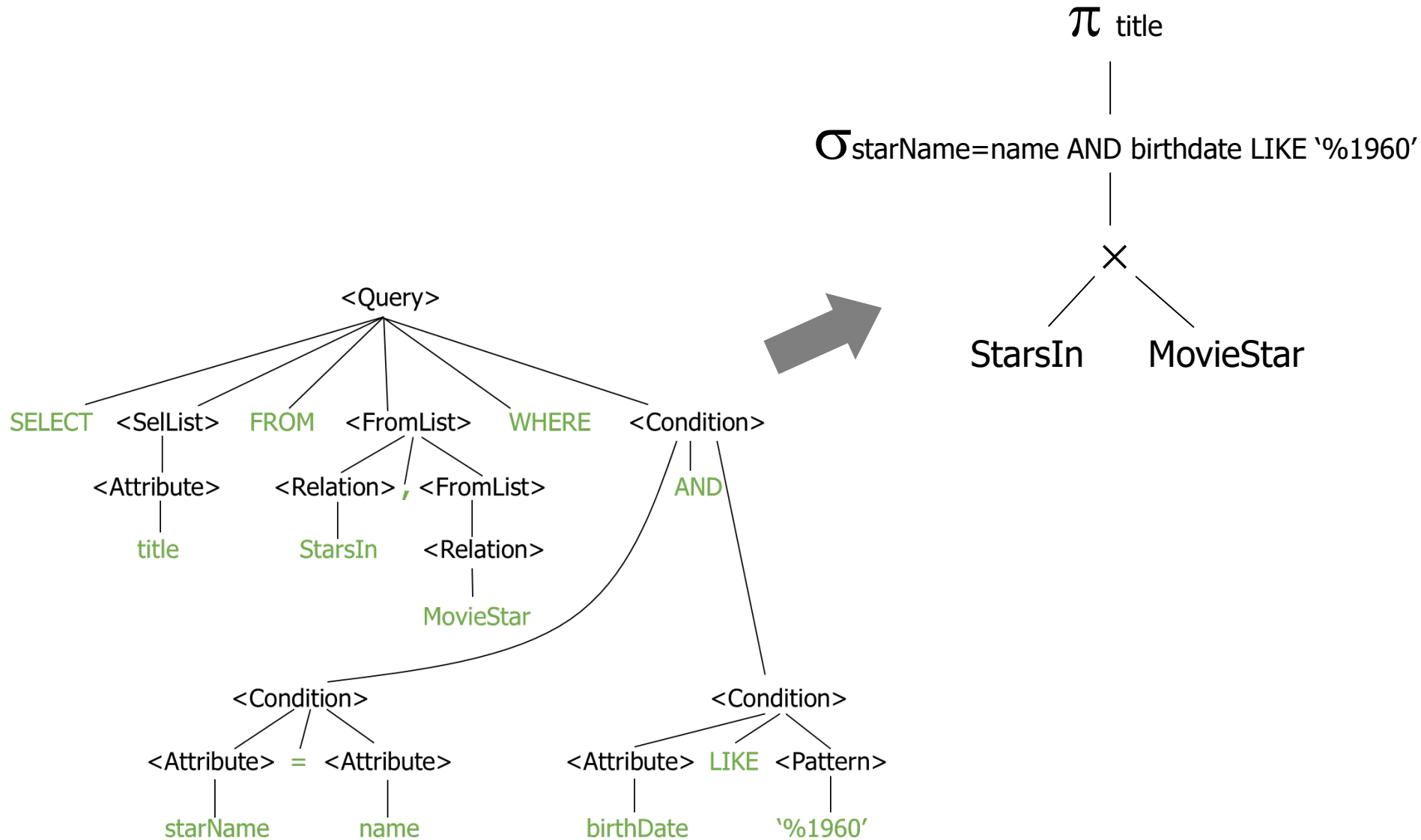
$$\gamma_L(R) = \gamma_L(\delta(R))$$



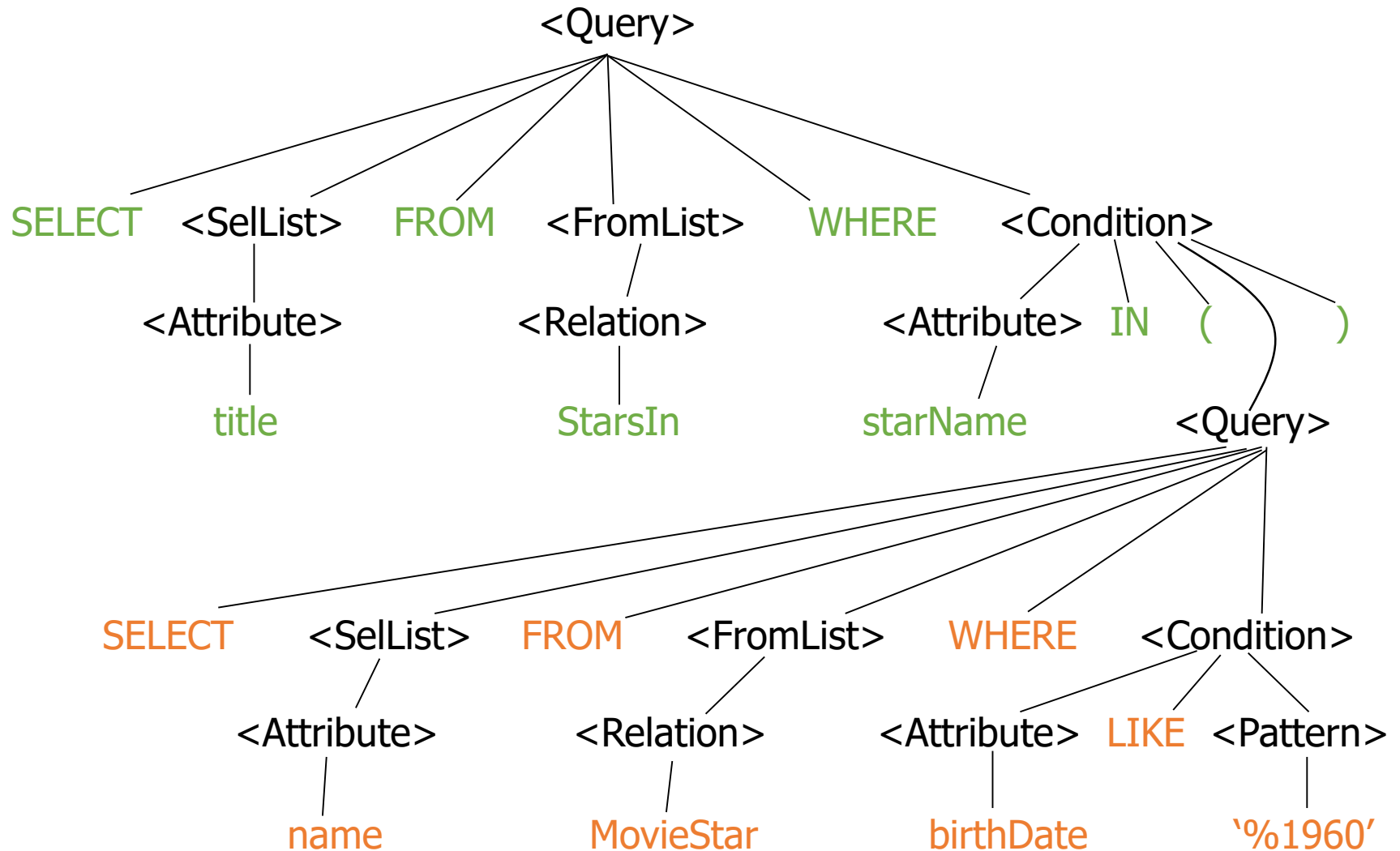
From parse trees to algebraic logical query plans

- If $\langle \text{Query} \rangle$ has a $\langle \text{Condition} \rangle$ of no subqueries, we may replace the tree from bottom to top as follows:
 - product of all the relations in $\langle \text{FromList} \rangle$
 - selection σ_C , where C is $\langle \text{Condition} \rangle$
 - projection π_L , where L is the list of attributes in $\langle \text{SelList} \rangle$

Example: parse tree to logical query plan

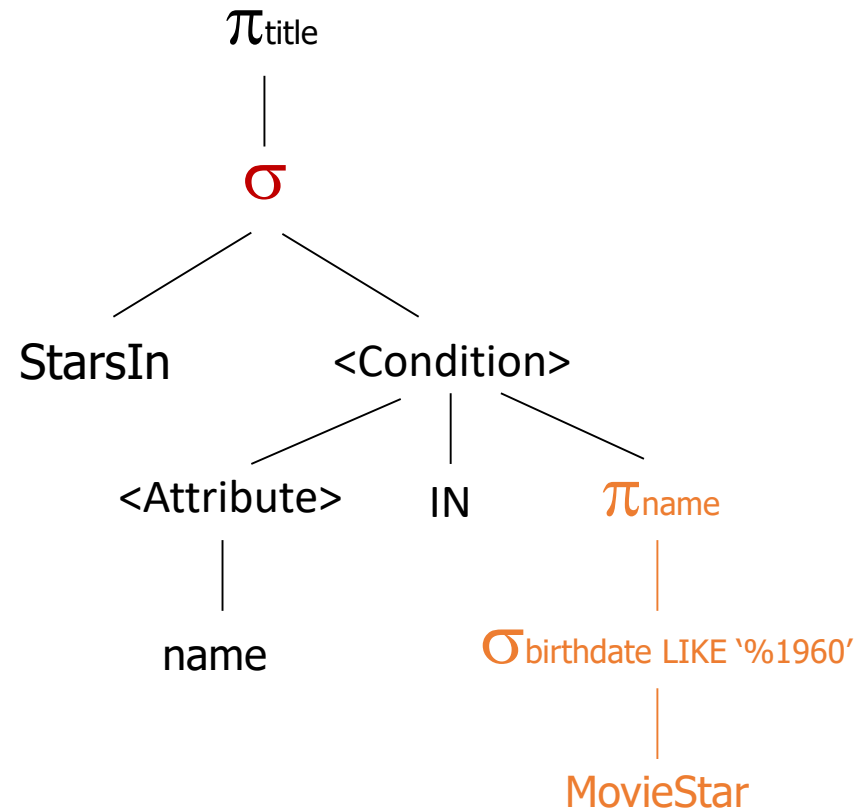
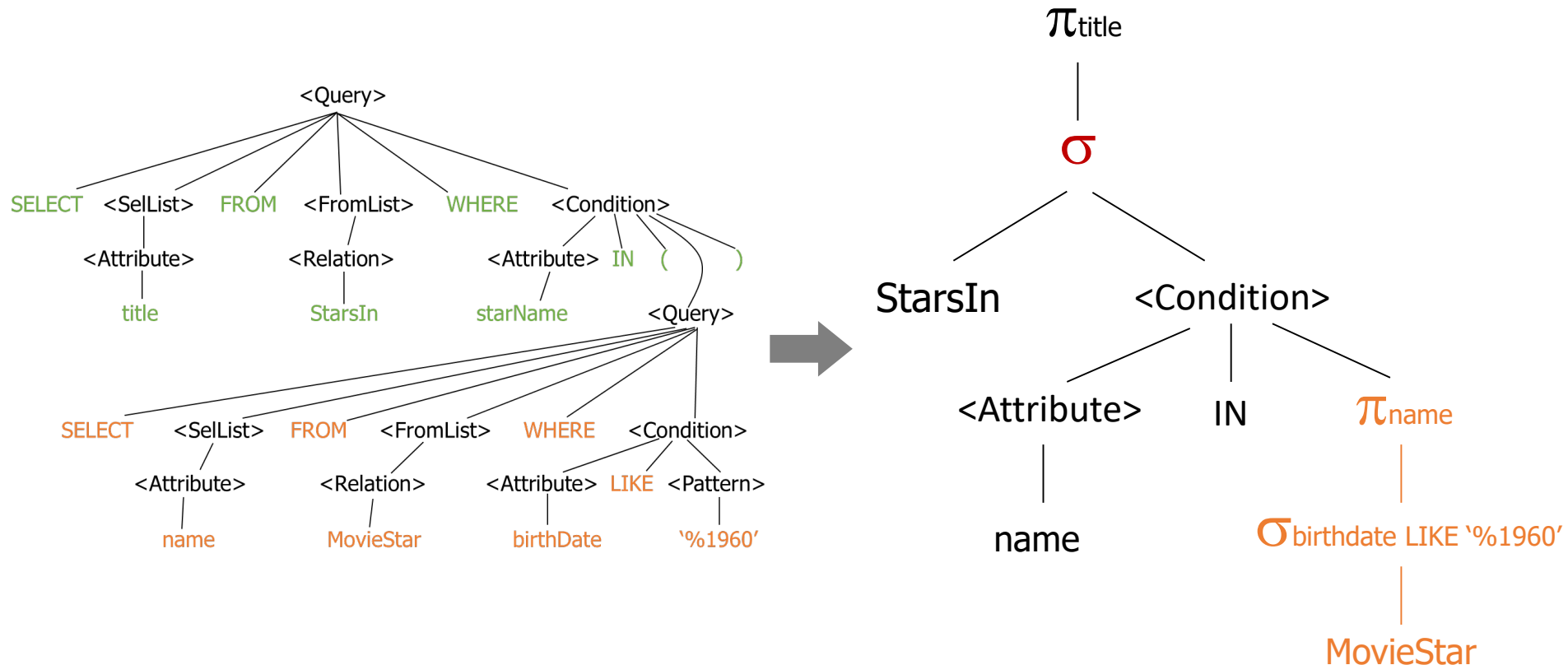


Example: Parse Tree (revisited)



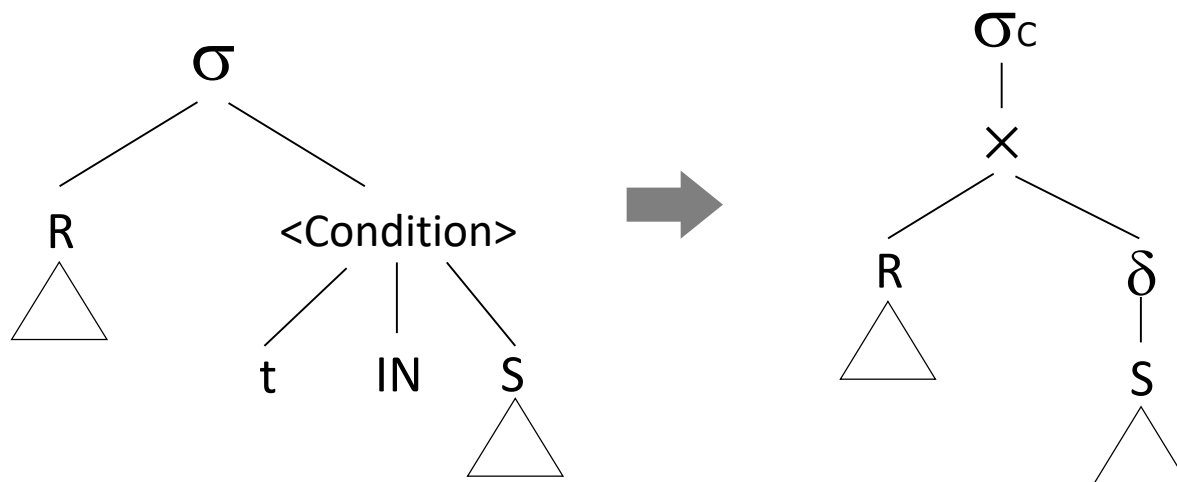
Removing subqueries from conditions

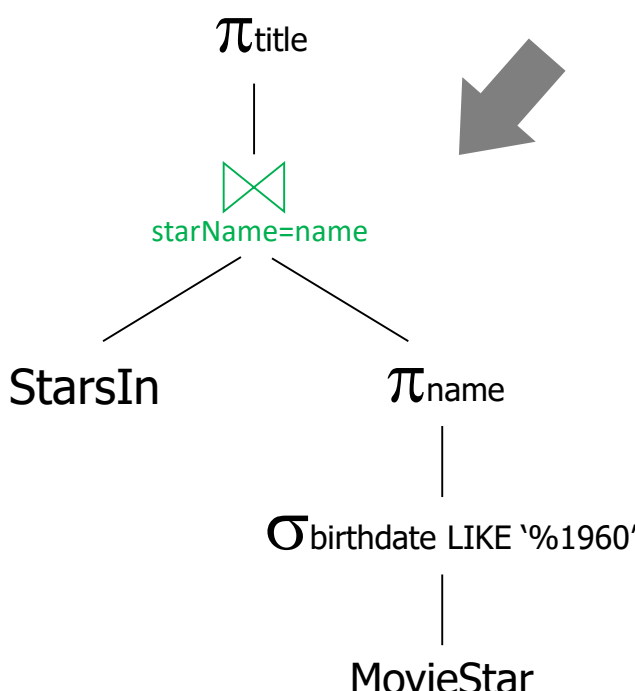
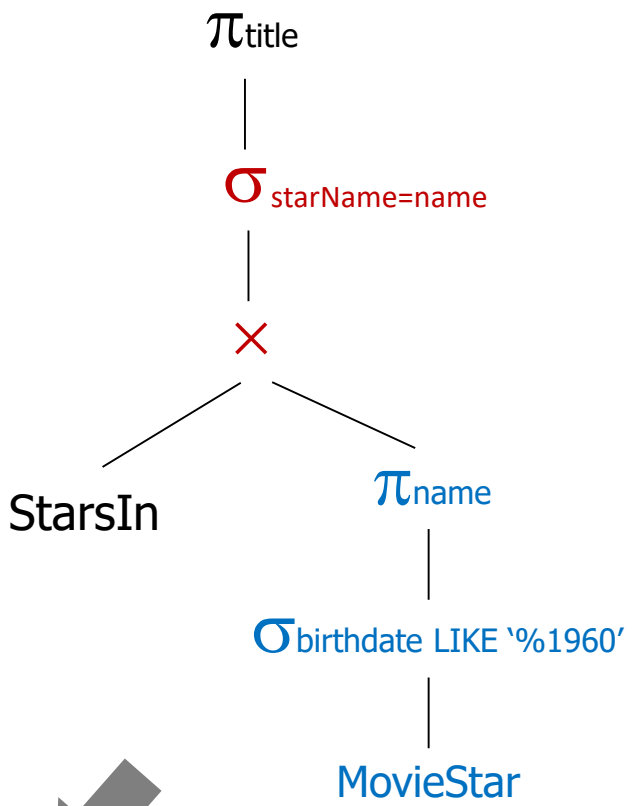
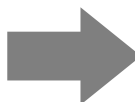
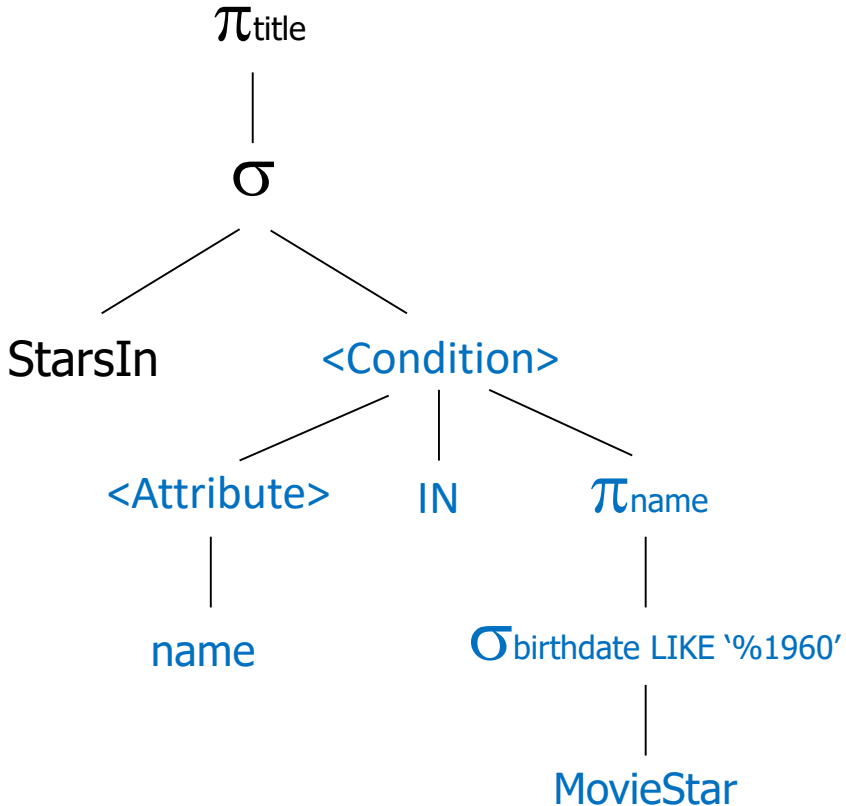
- **Two-argument selection** : σ with no parameter
 - left child: **relation R** on which the selection is being performed
 - right child: the **condition** applied to each tuple of R



Replacing two-argument selection

- Each form of condition may require its own rule
- **t IN S: uncorrelated subquery**
 - subquery's relation can be computed once and for all
 - ① replace <Condition> by the tree for S (may include δ)
 - ② replace two-argument selection by one-argument selection σ_C
 - C : the condition that equates t to the corresponding attribute of S
 - ③ give the product of R and S to the argument of σ_C

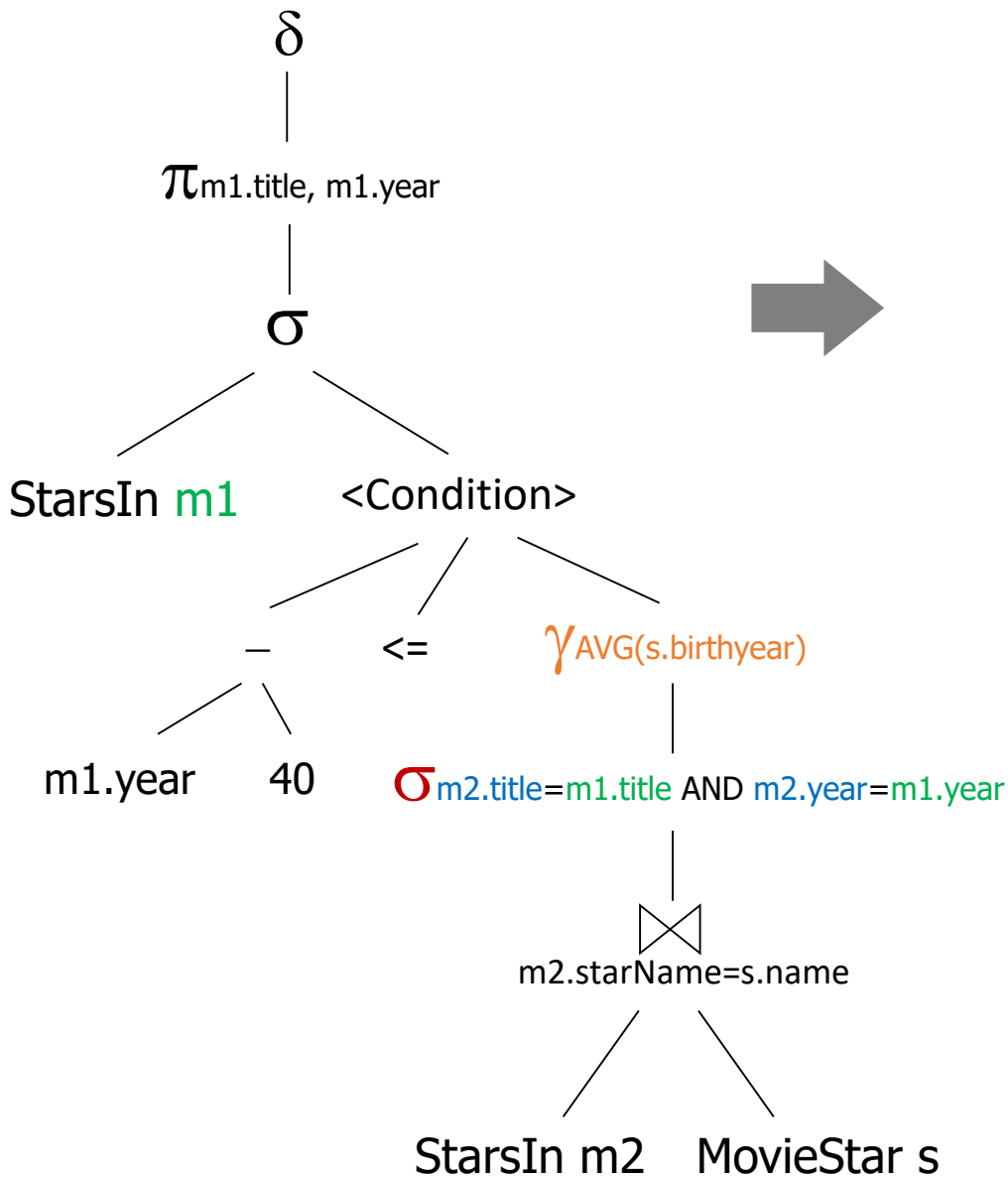


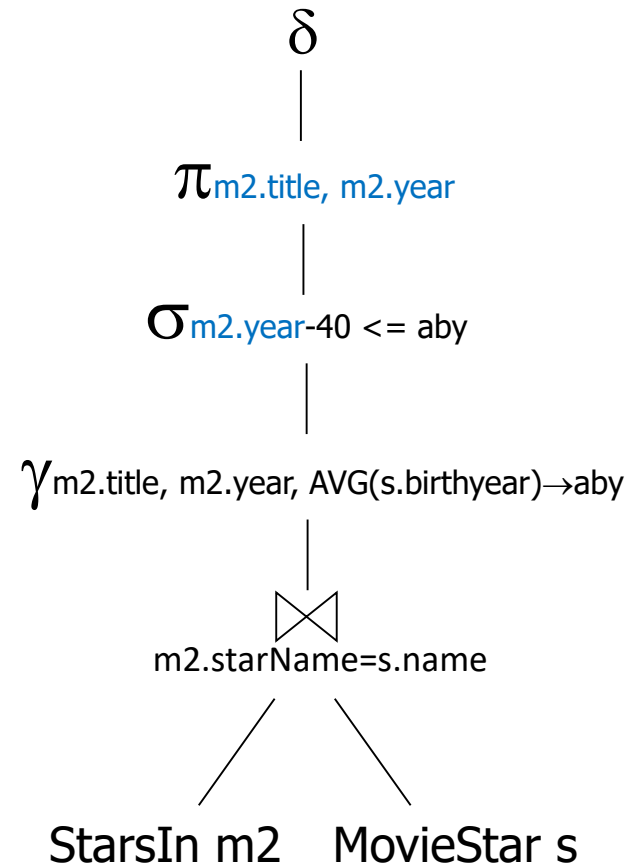
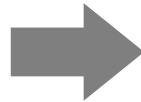
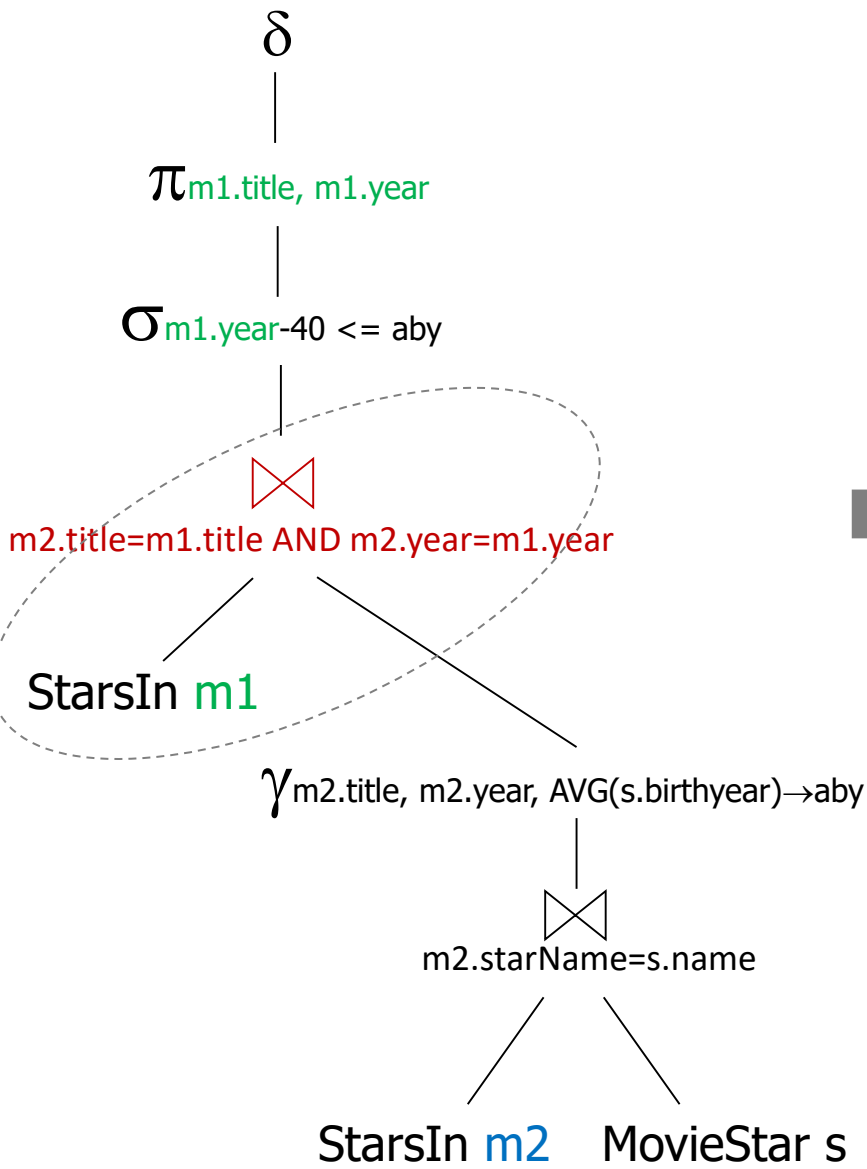


Translating correlated subqueries

- so as to produce a relation in which certain **extra attributes appear** that must **later be compared with the externally defined attributes**
 - must avoid introducing duplicate tuples
- e.g., find the movies where the average age of the stars was at most 40 when the movie was made

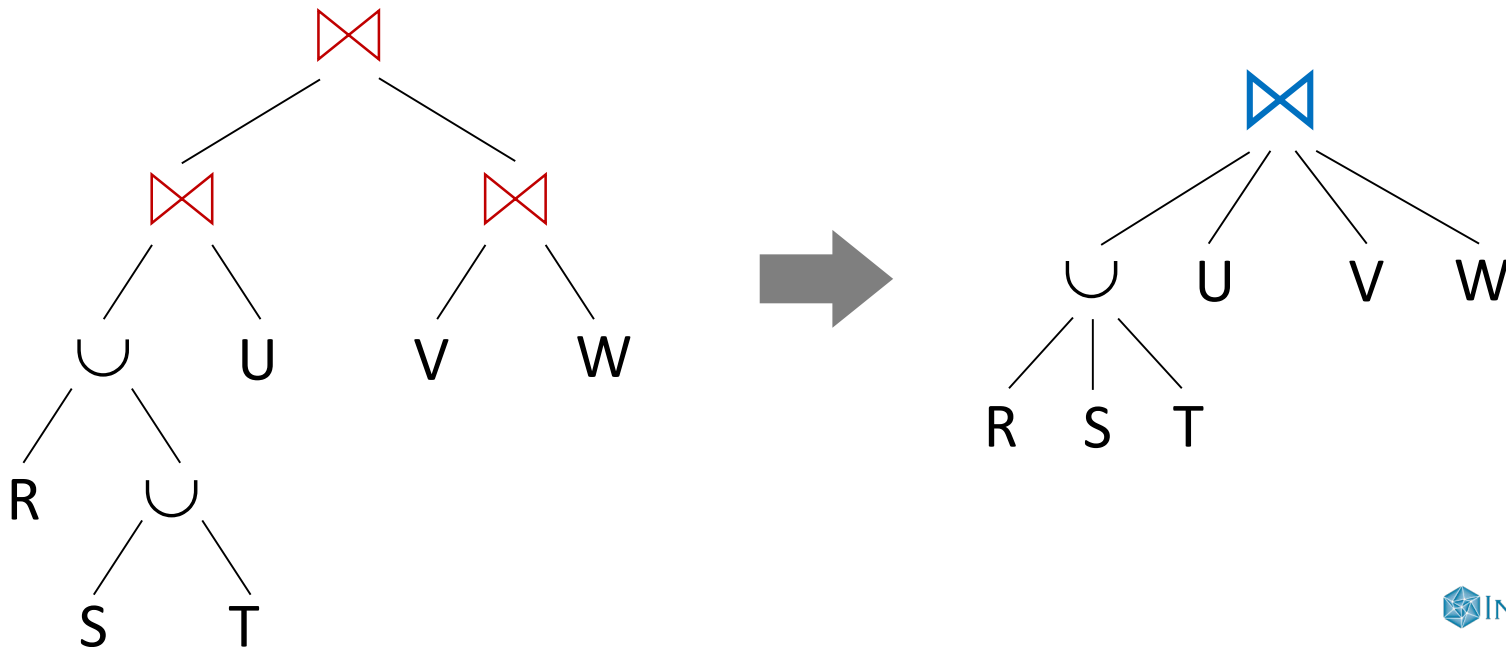
```
SELECT DISTINCT m1.title, m1.year
FROM StarsIn m1
WHERE m1.year - 40 <= (
    SELECT AVG(birthyear)
    FROM StarsIn m2, MovieStar s
    WHERE m2.starName = s.name AND
           m1.title = m2.title AND
           m1.year = m2.year
);
```





Grouping associative / commutative operators

- Associative and commutative operators may be thought of as having any number of operands
 - e.g., multiway join, multiway union, ...
 - multiway join is executed as a sequence of binary joins
 - let us reorder the operands so that they run faster than the original order specified by the parse tree



Cost Estimation

From logical plans to physical plans

- Many different physical plans from the logical plan

① Cost-based enumeration

- enumerate possible physical plans
- estimate the cost of each without executing it
- based on the parameters of the data (i.e., statistics)

② Pick the one with the least estimated cost

③ Pass the one to the query execution engine

Notation (remind)

- Size of intermediate relations: profound influence on costs
- $B(R)$: # of blocks needed to hold relation R
- $T(R)$: # of tuples of relation R
- $V(R,A)$: # of distinct values for attribute A of relation R

Estimating the size of projection

- Relation $R(a, b, c)$: $T(R) = 10,000$, $B(R) = 1250$
 - tuple size: 120 bytes
 - tuple header: 12 bytes
 - attributes a and b : 4 bytes (integer)
 - attribute c : 100 bytes (string)
 - block size: 1024 bytes
 - block header: 24 bytes
 - 8 tuples / block
- e.g., $S = \pi_{a+b \rightarrow x, c}(R)$, $B(S) = 1250$
 - each tuple of S : $12(\text{header}) + 4(x) + 100(c) = 116$ bytes
- e.g., $U = \pi_{a, b}(R)$, $B(U) = 200$
 - each tuple of U : $12(\text{header}) + 4(a) + 4(b) = 20$ bytes
 - 50 tuples / block

Estimating the size of selection (S is a result)

- $T(S) = T(R) / V(R,A)$ for equality comparison
- $T(S) = T(R) / 3$ for inequality comparison
- $T(S) = n \left(1 - \left(1 - \frac{m1}{n} \right) \left(1 - \frac{m2}{n} \right) \right)$ for $S = \sigma_{\substack{c1 \\ \downarrow \\ m1}} \text{ OR } \substack{c2 \\ \downarrow \\ m2} (R)$
 $\substack{n \\ \downarrow \\ n}$
- e.g., $T(R) = 10,000$, $V(R,a) = 50$
 - $T(S) = \sigma_{a=10}(R) = 10000 / 50 = 200$
 - $T(S) = \sigma_{b<20}(R) = 10000 / 3 = 3333$
- e.g., $S = \sigma_{a=10 \text{ OR } b<20}(R)$, $T(S) = 3466$

$$10000 \left(1 - \left(1 - \frac{200}{10000} \right) \left(1 - \frac{3333}{10000} \right) \right) = 3466$$
- e.g., $S = \sigma_{a=10 \text{ AND } b<20}(R) = 10000 \left(\left(\frac{200}{10000} \right) \left(\frac{3333}{10000} \right) \right) = 67$

Estimating the size of natural join (single attr.)

- Two key assumptions for $R(X,Y) \bowtie S(Y,Z)$
 - Assumption 1 (**containment**-of-value-sets)
 $V(R,Y) \leq V(S,Y)$, then every Y-value of R will be a Y-value of S
 - Assumption 2 (**preservation**-of-value-sets)
A is an attribute of R, but not of S, then $V(R \bowtie S, A) = V(R, A)$

Estimating the size of natural join (single attr.)

- Case 1: $V(R,Y) \geq V(S,Y)$
 - Y-value of $s \in S$ is surely one of the Y-values of R (by containment assumption)
 - expected number of tuples in S that $r \in R$ matches: $T(S) / V(R,Y)$
 - e.g., $T(R) = 100$, $V(R,Y) = 100$, $T(S) = 50$, $V(S,Y) = 50$,
 - each $r \in R$ matches $50 \times 1/100 = 0.5$ tuple(s) in S
 - so, the size of the join result is estimated as $T(R) \times T(S) / V(R, Y)$
- Case 2: $V(R,Y) \leq V(S,Y)$
 - expected number of tuples in R that $s \in S$ matches: $T(R) / V(S,Y)$
 - the size of the join result is estimated as $T(S) \times T(R) / V(S, Y)$

$$T(R \bowtie S) = \frac{T(R)T(S)}{\max(V(R, Y), V(S, Y))}$$

Natural join among three relations (1)

$R(a,b)$	$S(b,c)$	$U(c,d)$
$T(R) = 1000$	$T(S) = 2000$	$T(U) = 5000$
$V(R,b) = 20$	$V(S,b) = 50$	
	$V(S,c) = 100$	$V(U,c) = 500$

- Plan 1: $(R \bowtie S) \bowtie U$

- $T(R \bowtie S) = 1000 \times 2000 / 50 = 40,000$
- $T((R \bowtie S) \bowtie U) = T(R \bowtie S)T(U) / \max(V(R \bowtie S, c), V(U, c))$
 $= 40,000 \times 5000 / \max(100, 500) = 400,000$

by preservation assumption

- Plan 2: $R \bowtie (S \bowtie U)$

- $T(S \bowtie U) = 2000 \times 5000 / 500 = 20,000$
- $T(R \bowtie (S \bowtie U)) = T(R)T(S \bowtie U) / \max(V(R, b), V(S \bowtie U, b))$
 $= 1000 \times 20,000 / \max(20, 50) = 400,000$

Natural joins with multiple join attributes

- $R(a,b,c) \bowtie_{R.b=S.d \text{ AND } R.c=S.e} S(d,e,f)$

$R(a,b,c)$	$S(d,e,f)$
$T(R) = 1000$	$T(S) = 2000$
$V(R,b) = 20$	$V(S,d) = 50$
$V(R,c) = 100$	$V(S,e) = 50$

- $T(R(a,b,c) \bowtie_{R.b=S.d \text{ AND } R.c=S.e} S(d,e,f))$

$$= \frac{1000 \times 2000}{\max(20, 50) \times \max(100, 50)} = 400$$

Natural join among three relations (2)

R(a,b)	S(b,c)	U(c,d)
T(R) = 1000	T(S) = 2000	T(U) = 5000
V(R,b) = 20	V(S,b) = 50	
	V(S,c) = 100	V(U,c) = 500

- $T(R(a,b) \bowtie U(c,d)) = 1000 \times 5000 = 5,000,000$
- Plan 3: $T(\underbrace{(R(a,b) \bowtie U(c,d))}_{50 \times 500} \bowtie S(b,c)) = \frac{5000000 \times 2000}{50 \times 500} = 400,000$
- The same size regardless of join orders

Joins among many relations

- $S = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$

$R(a,b,c)$	$S(b,c,d)$	$U(b,e)$
$T(R) = 1000$	$T(S) = 2000$	$T(U) = 5000$
$V(R,a) = 100$		
$V(R,b) = 20$	$V(S,b) = 50$	$V(U,b) = 200$
$V(R,c) = 200$	$V(S,c) = 100$	
	$V(S,d) = 400$	
		$V(U,e) = 500$

→ divide by the two largest $V(:,b)$

→ divide by the largest $V(:,c)$

- $$T(S) = \frac{1000 \times 2000 \times 5000}{50 \times 200 \times 200} = 5000$$

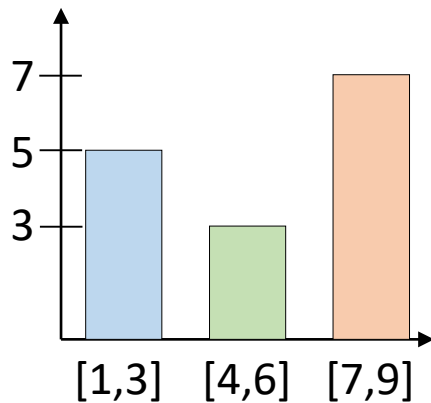
Estimating sizes for other operations

- Union
 - bag: sum of the sizes of the arguments
 - set: midpoint of [size of larger one (i.e., min), sum of both sizes (i.e., max)]
- Intersection
 - midpoint of [0, size of smaller one]
- Difference: $R - S$
 - $[T(R), T(R) - T(S)]$
 - estimation suggested: $T(R) - T(S)/2$
- Duplication elimination : $R(a_1, a_2, \dots, a_n)$
 - we usually do not have $V(R, [a_1, a_2, \dots, a_n])$
 - smaller one between $T(R)/2$ and the product of all the $V(R, a_i)$'s ($1 \leq i \leq n$)
- Grouping and aggregation : $\gamma_L(R)$, $L = \{g_1, g_2, \dots, g_k\}$
 - smaller one between $T(R)/2$ and the product of all the $V(R, g_i)$'s ($1 \leq i \leq k$)

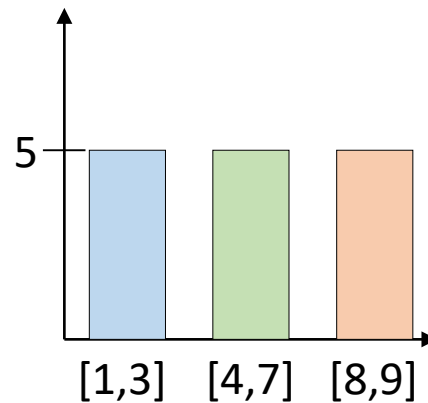
Cost-based Plan Selection

Histogram

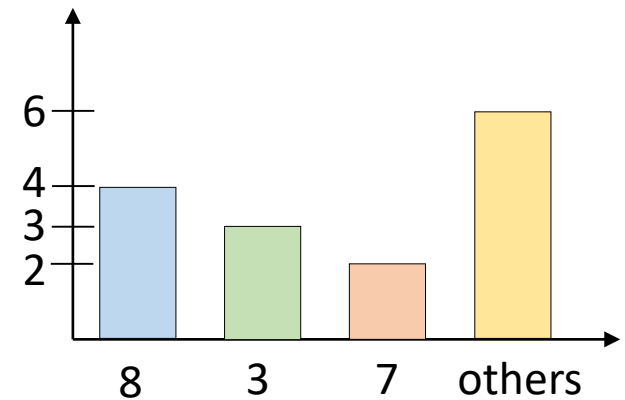
- Statistics about $V(R,a)$
 - e.g., $\{1, 2, 3, 3, 3, 4, 5, 6, 7, 7, 8, 8, 8, 8, 9\}$



equal-width



equal-height



most-frequent-values

Example of estimation using histogram (1)

- $R(a,b) \bowtie S(b,c)$ $V(R,b) = 14, \quad V(S,b) = 13$
- The most-frequent-values histogram
 - histogram($R.b$) = [1: 200, 0: 150, 5: 100, others(11 values): 550]
 - histogram($S.b$) = [0: 100, 1: 80, 2: 70, others(10 values): 250]
 - # of tuples for $b = 0$: $150 \times 100 = 15,000$
 - # of tuples for $b = 1$: $200 \times 80 = 16,000$
 - # of tuples for $b = 2$: $50 \times 70 = 3,500$
 - # of tuples for $b = 5$: $100 \times 25 = 2,500$
 - # of tuples for nine b 's: $50 \times 25 \times 9 = 11,250$

 - total # of tuples 48,250

50 on average

25 on average

Example of estimation using histogram (2)

- Jan(day, temp) : 245 tuples
- July(day, temp) : 245 tuples

```
SELECT Jan.day, July.day
FROM Jan, July
WHERE Jan.temp=July.temp;
```

- Estimation w/ histogram

- [40-49]: $10 \times 5 / 10 = 5$
- [50-59]: $5 \times 20 / 10 = 10$
- result size: 15

width of range

- Estimation w/o histogram

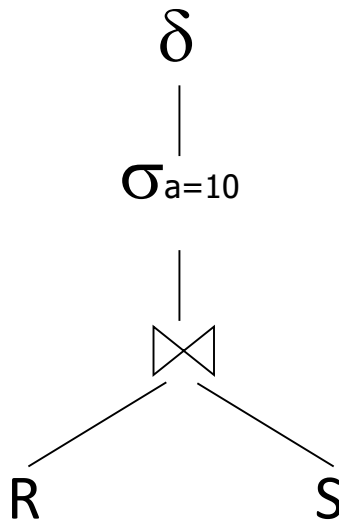
- $V(\text{Jan, temp}) = V(\text{July, temp}) = 100$
- result size: $245 \times 245 / 100 = 600$

range	Jan	July
0-9	40	0
10-19	60	0
20-29	80	0
30-39	50	0
40-49	10	5
50-59	5	20
60-69	0	50
70-79	0	100
80-89	0	60
90-99	0	10

equal-width histogram

Reducing the cost of logical query plans

- Estimate both **costs of before and after a transformation**
 - allow us to know where a transformation appears to reduce cost
- Compute the **sum of the sizes of all intermediate results**
 - **not include the size of output** (in Chapter 15)
 - cannot be based on disk I/O's in logical query plans



Example

$R(a,b)$	$S(b,c)$	$W(a,b,c)$
$T(R) = 5000$ $V(R,a) = 50$ $V(R,b) = 100$	$T(S) = 2000$ $V(S,b) = 200$ $V(S,c) = 100$	$V(W,a) = 1$ $V(W,b) = 100$ $V(W,c) = 100$

size of $\delta = \min(\underbrace{T(R)/2}_{=50}, \underbrace{V(R,a)}_{=1} \cdot \underbrace{V(R,b)}_{=100})$

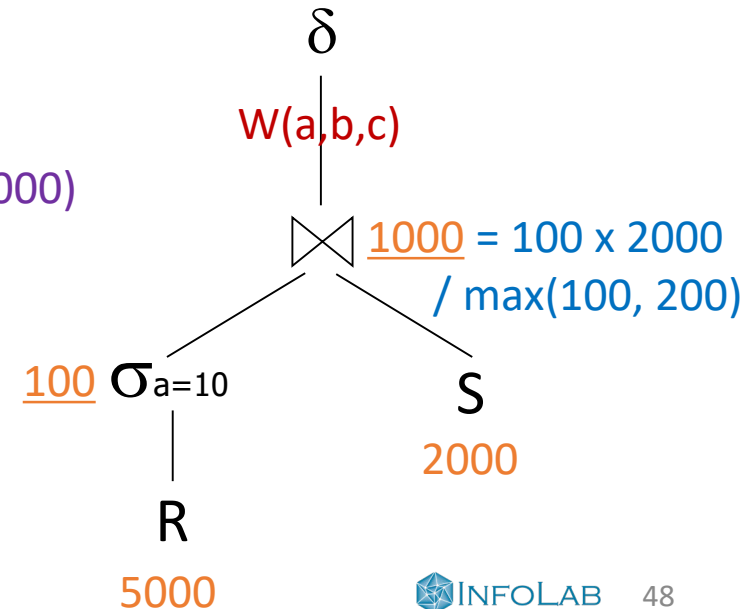
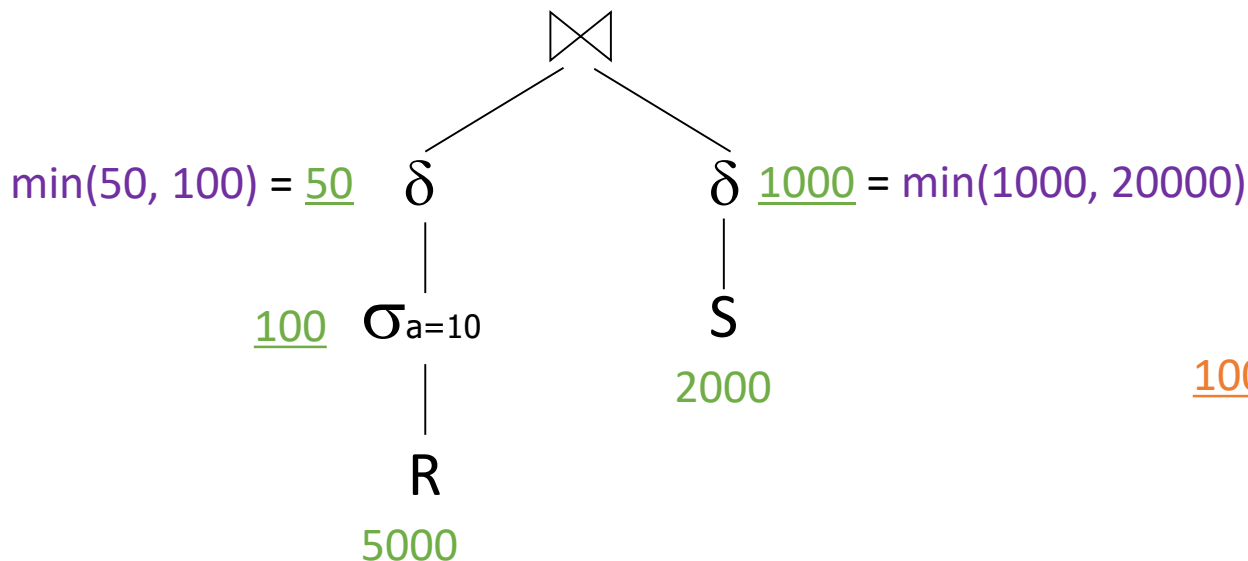
Plan 1 total: **1150**

>

total: **1100** **Plan 2**

$250 = 50 \times 1000 / \max(100, 200)$

$500 = \min(500, 10000)$



Enumerating physical plans

- Baseline approach: **exhaustive enumeration**
- Two approaches to exploring the space of possible plans
- **Two-down**
 - consider each possible way to evaluate the argument(s) for each possible implementation of an operation from the root
- **Bottom-up**
 - compute the costs of all possible ways to compute each subexpression of the tree
 - **dynamic programming**
 - **Selinger-style** (or **System-R-style**)

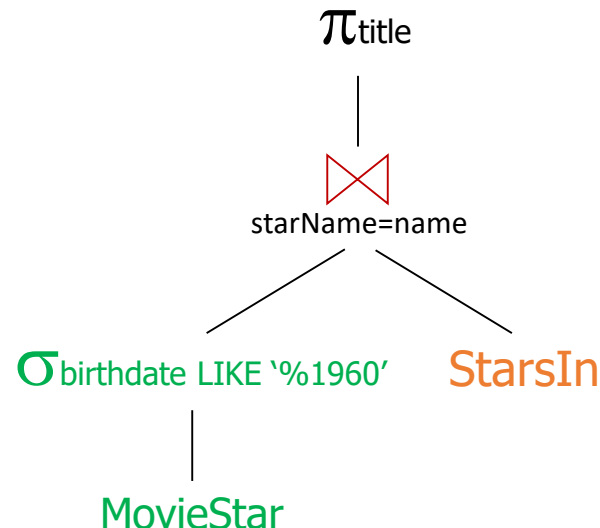
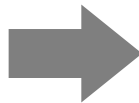
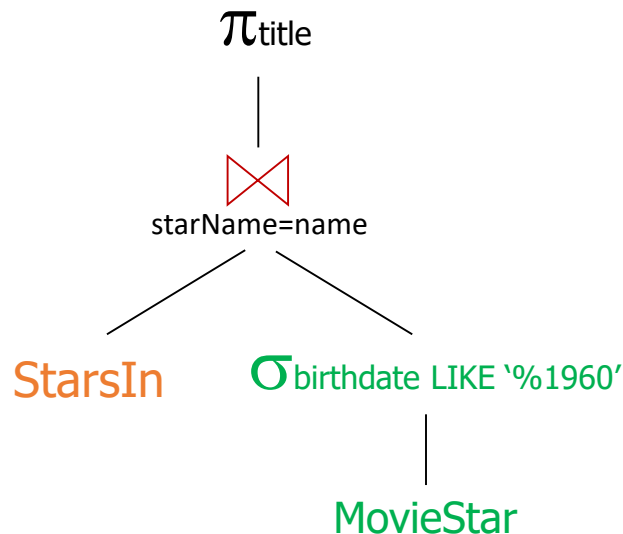
Choosing order for joins

- Selecting an order for the join of three or more relations
 - critical problem in cost-based optimization
- Significance of left and right join arguments
 - many of join methods are asymmetric
 - cost of the join depends on which relation plays which role (i.e., left or right argument)
 - one-pass join: build relation (smaller), probe relation (larger)
 - nested-loop join: outer loop (left), inner loop (right)
 - index join: right argument has the index

Join trees

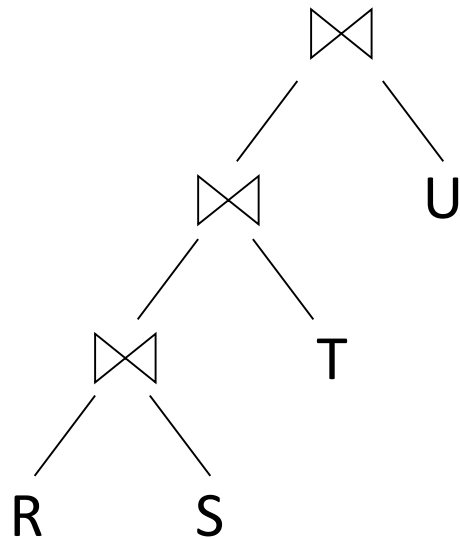
- Select the relation of a **smaller** estimated size as the **left** argument conventionally

```
SELECT title
FROM StarsIn, MovieStar
WHERE starName = name AND
      birthdate LIKE '%1960';
```

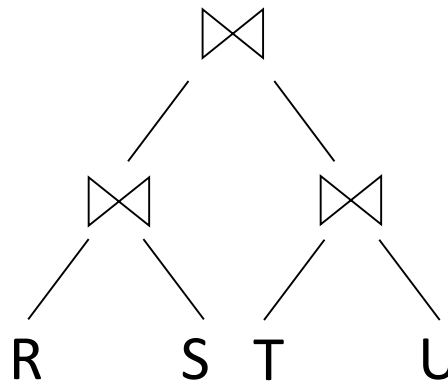


Shapes of join trees

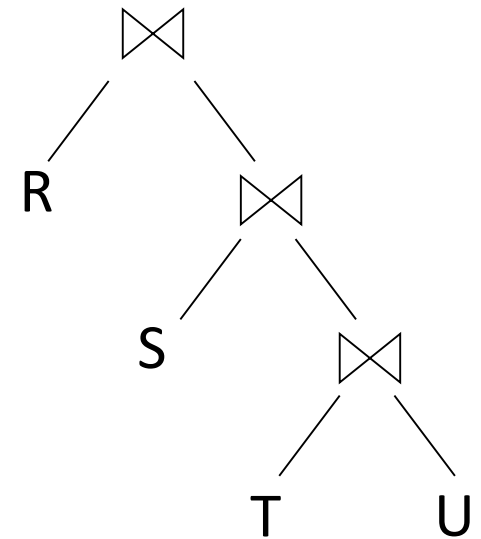
- **Left-deep**: a binary tree all of whose right children are leaves
- **Right-deep**: a binary tree all of whose left children are leaves
- **Bushy**: neither left-deep nor right-deep



left deep



bushy



right deep

Number of possible join trees: $T(n) \times n!$

- Two choices for a join tree for two relations
- $n!$ choices for a left(or right)-deep join tree for n relations
- $T(n)$: total number of tree shapes

$$T(1) = 1$$

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i) \quad , n > 1$$

n	1	2	3	4	5	6
T(n)	1	1	2	5	14	42

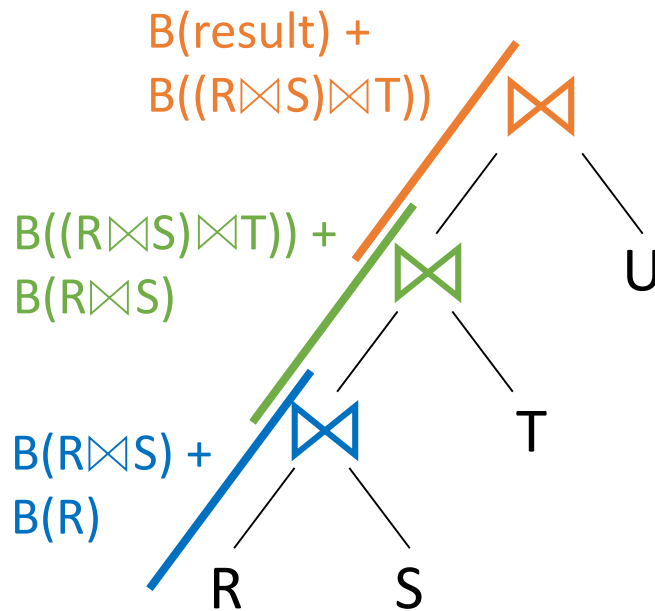
- e.g., $n=6$, # of left-deep join trees = $6! = 720$,
of all possible join trees = $42 \times 6! = 30240$

Two-fold advantages of only using left-deep

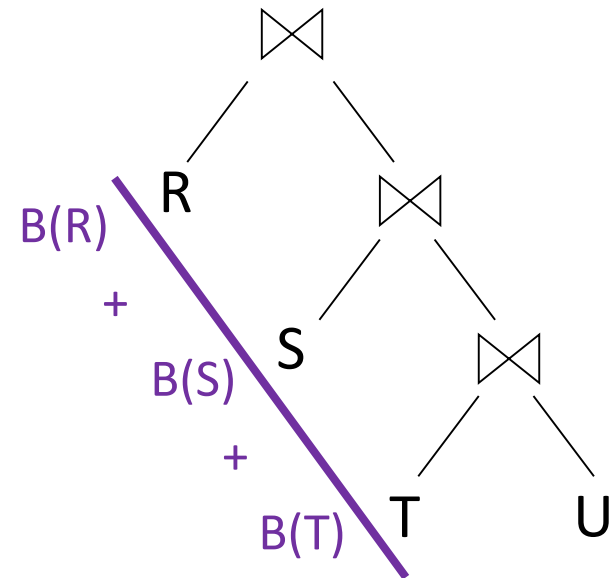
- Reducing the search space
 - # of possible left-deep trees (i.e., $n!$) is much smaller than # of all possible trees (i.e., $T(n) \times n!$)
- Better suited for common join algorithms
 - e.g., nested-loop joins, one-pass joins, ...
 - more efficient than the same algorithms used with non-left-deep trees

Example: **memory usage** for one-pass joins

- Assumption: $|R| < |S| < |T| < |U|$, **materialization**



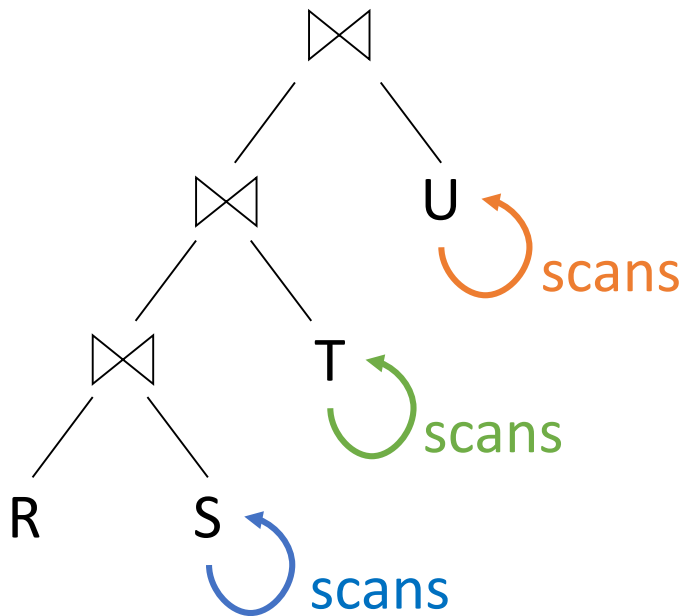
left deep



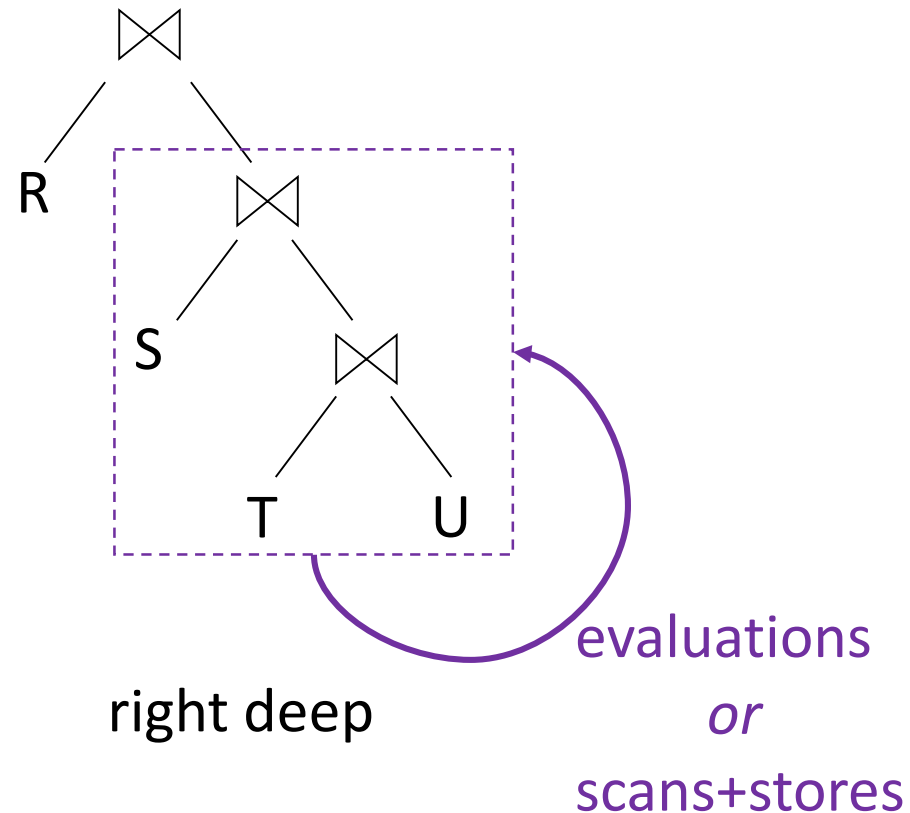
right deep

Example: # of scans for nested-loop joins

- Assumption: $|R| < |S| < |T| < |U|$, **pipelining**



left deep



right deep

Dynamic programming: BASIS

- $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$ (# of possible left-deep join trees: $n!$)
- BASIS
 - cost (entry in a DP table) : size of intermediate relations
 - a single relation R : 0
 - a pair of relations $\{R_i, R_j\}$: 0 (no intermediate relations)
 - (result) size: calculated as in “Joins of many relations” (p.41)
 - a single relation R : $B(R)$ or $T(R)$
 - best join order (best plan)
 - a single relation R : R
 - a pair of relations: a smaller relation as the left argument

Example of dynamic programming

- Four relations (R, S, T, and U) having 1000 tuples

BASIS (single relation)

	{R}	{S}	{T}	{U}
size	1000	1000	1000	1000
cost	0	0	0	0
best plan	R	S	T	U

R(a,b)	S(b,c)	T(c,d)	U(d,a)
V(R,a) = 100			V(U,a) = 50
V(R,b) = 200	V(S,b) = 100		
	V(S,c) = 500	V(T,c) = 20	
		V(T,d) = 50	V(U,d) = 1000

1000 x 1000 / max(200, 100)



BASIS (a pair of relations)

	{R,S}	{R,T}	{R,U}	{S,T}	{S,U}	{T,U}
size	5000	1,000,000	10,000	2000	1,000,000	1000
cost	0	0	0	0	0	0
best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

Both relations are the same size, and so, take the first in alphabetical order to be the left argument

Dynamic programming: INDUCTION

- We consider only left-deep join trees
- INDUCTION: $\mathbf{R} = \{R_1, \dots, R_k\}$
 - consider $\mathbf{R} - \{R_i\}$ as left and R_i as right (in the root node)
 - $\text{cost}(\mathbf{R}) = \text{cost}(\mathbf{R} - \{R_i\}) + \text{size}(\mathbf{R} - \{R_i\})$
 - pick the best R_i among $\{R_1, \dots, R_k\}$ having the least $\text{cost}(\mathbf{R})$

BASIS (a pair of relations)

	{R,S}	{R,T}	{R,U}	{S,T}	{S,U}	{T,U}
size	5000	1,000,000	10,000	<u>2000</u>	1,000,000	1000
cost	0	0	0	0	0	0
best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

INDUCTION (only left-deep trees, three relations)

	{R,S,T}	{R,S,U}	{R,T,U}	{S,T,U}
size				
cost	2000			
best plan	$(S \bowtie T) \bowtie R$			

three possible partitions: $\{R,S\} + \{T\}$, $\{R,T\} + \{S\}$, $\{S,T\} + \{R\}$

cost of partition 1: $\text{cost}(\{R,S\}) + \text{size}(\{R,S\}) = 0 + 5,000 = 5,000$

cost of partition 2: $\text{cost}(\{R,T\}) + \text{size}(\{R,T\}) = 0 + 1,000,000 = 1,000,000$

cost of partition 3: $\text{cost}(\{S,T\}) + \text{size}(\{S,T\}) = 0 + 2,000 = 2,000$ ($\leftarrow$ best)

$R(a,b)$	$S(b,c)$	$T(c,d)$	$U(d,a)$
$V(R,a) = 100$			$V(U,a) = 50$
$V(R,b) = 200$	$V(S,b) = 100$		
	$V(S,c) = 500$	$V(T,c) = 20$	
		$V(T,d) = 50$	$V(U,d) = 1000$

INDUCTION (only left-deep trees, three relations)

	$\{R,S,T\}$	$\{R,S,U\}$	$\{R,T,U\}$	$\{S,T,U\}$
size	10,000	50,000	10,000	2000
cost	2000	5000	1000	1000
best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

three possible partitions: $\{R,S\} + \{T\}$, $\{R,T\} + \{S\}$, $\{S,T\} + \{R\}$

cost of partition 3: $\text{cost}(\{S,T\}) + \text{size}(\{S,T\}) = 0 + 2,000 = 2,000$ ($\leftarrow$ best)

size of partition 3: $2000 \times 1000 / \max(200, 100) = 10,000$

	{R,S}	{R,T}	{R,U}	{S,T}	{S,U}	{T,U}
size	5000	1,000,000	10,000	2000	1,000,000	1000
cost	0	0	0	0	0	0
best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

	{R,S,T}	{R,S,U}	{R,T,U}	{S,T,U}
size	10,000	50,000	10,000	2000
cost	2000	5000	1000	1000
best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

type	partition	cost
left-deep	$((S \bowtie T) \bowtie R) \bowtie U$	12,000
	$((R \bowtie S) \bowtie U) \bowtie T$	55,000
	$((T \bowtie U) \bowtie R) \bowtie S$	11,000
	$((T \bowtie U) \bowtie S) \bowtie R$	3,000
bushy	$(T \bowtie U) \bowtie (R \bowtie S)$	6,000
	$(R \bowtie T) \bowtie (S \bowtie U)$	2,000,000
	$(S \bowtie T) \bowtie (R \bowtie U)$	12,000

← 2000 + 10000

← best plan!

← 1000 + 5000

Principle issues for completing physical query plan

- ① Select the algorithms to implement the operations of the query plan
 - we have just done selection of a join order by DP
- ② Decide whether intermediate results are materialized or pipelined
 - materialization: create the whole result and store it on disk or in main memory
 - pipelining: create a part of the result in main memory
- ③ Notation: physical-query-plan operators must include details regarding access methods and algorithms

Choosing a selection method for $\sigma_C(R)$

- Assumption: C is **ANDs** of conditions
 - at least one condition: $A\theta c$
 - A: attribute with an index
 - c: constant
 - θ : comparison operator (e.g., =, <, >)
- Selection operation is done in **two steps** (for simplicity)
 - ① retrieve all tuples **satisfying a comparison**
 - using the **index-scan** physical operator, or
 - read the entire relation using the **table scan** physical operator
 - ② decide whether each tuple selected **satisfies the rest of the selection condition** (called **filter operator**)

- Measure of cost: **disk I/O's**
 - count only the cost of accessing data blocks, not index blocks
- Three cases for cost estimation of selection
 - **table-scan**
 - $B(R)$ if R is clustered
 - $T(R)$ otherwise
 - **index-scan for an equality** term (e.g., $a=10$)
 - $B(R)/V(R,a)$ if the index is clustering
 - $T(R)/V(R,a)$ otherwise
 - **index-scan for an inequality** term (e.g., $b < 20$)
 - $B(R)/3$ if the index is clustering
 - $T(R)/3$ otherwise

Example: $\sigma_{x=1 \text{ AND } y=2 \text{ AND } z<5} (R)$ for $R(x, y, z)$

- Parameters

- $T(R) = 5000$, $B(R) = 200$

- $V(R,x) = 100$, $V(R,y) = 500$

- R is clustered

- indexes on all x , y , and z , but only the index on z is clustering

① Table-scan (followed by filter): 200 disk I/O's

② Index-scan on x : 50 disk I/O's

- $T(R) / V(R,x) = 50$ tuples with $x = 1$, and index is non-clustering

③ Index-scan on y : 10 disk I/O's (← chosen as the best)

- $T(R) / V(R,y) = 10$ tuples with $y = 2$, and index is non-clustering

④ Index-scan on z : 67 disk I/O's ($B(R) / 3 = 67$)

Choosing a join method (algorithm)

- If we know # of buffers available, use the cost in Chap.15
- Five guidelines
 - if we do not know # of buffers available, or not have important statistics (e.g., $V(R,a)$)
- ① Call for one-pass
 - hoping that buffer manager can devote enough buffers to the join or come close (thrashing is not a major cost in this case)
- ② Call for nested-loop
 - hoping that the left argument will not have to be divided into too many pieces

③ Sort-join is a good choice when

- one or both arguments are already sorted on join attribute(s)
- two or more joins on the same attribute as below

$$(R(\textcolor{red}{a},b) \bowtie S(\textcolor{red}{a},c)) \bowtie T(\textcolor{red}{a},d)$$

(the result of $R \bowtie S$ can be used directly in the next sort-join)

④ Index-join is good for $R(a,b) \bowtie S(b,c)$ when

- R is expected to be small (perhaps due to a selection on a key)
- an index exists on the join attribute $S.b$

⑤ Hash-join is good when

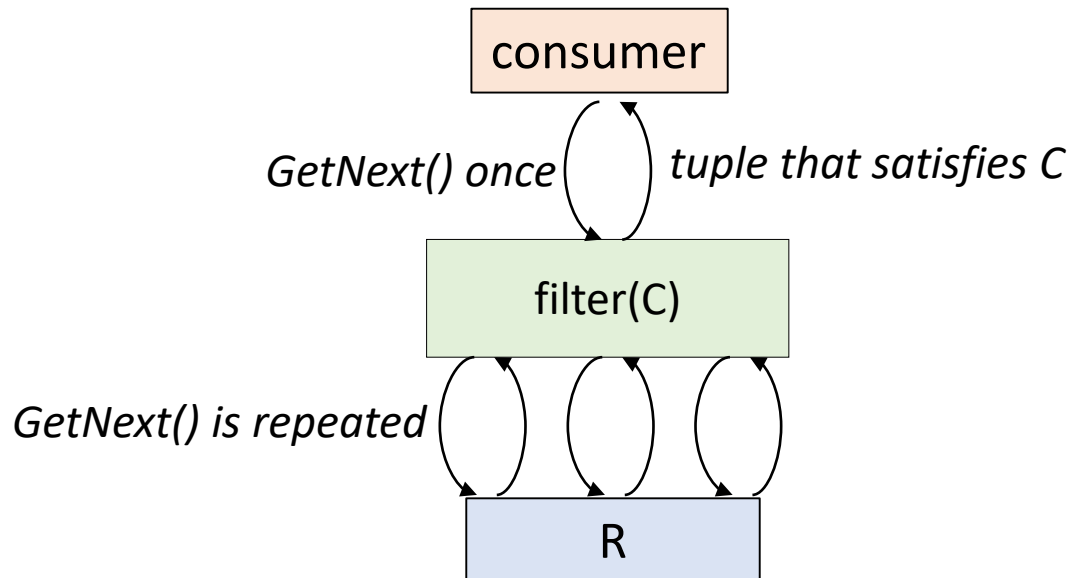
- no opportunity to use already-sorted relations or indexes
- probably the best choice in modern DBMS's
(due to the characteristics of modern computer H/Ws)

Pipelining vs. materialization

- Pipelining
 - passing the tuples produced by one operation directly to the operation that uses it
 - without ever storing the intermediate tuples on disk
 - e.g., stream data processing
- Advantage: saving disk I/O's
- Disadvantages
 - several operations must share main memory at any time
 - chance that the algorithms of higher disk I/O's must be chosen, or trashing will occur

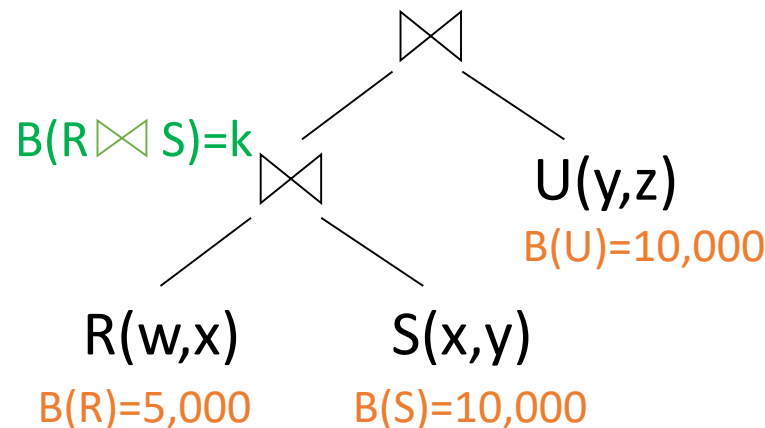
Pipelining unary operations

- Excellent candidates for pipelining (**tuple-at-a-time**)
 - e.g., selection, projection
- e.g., σ_C (or **filter(C)**)
 - consumer of the pipelined result calls `GetNext()` each time another tuple is needed
 - `filter(C)` **calls `GetNext()` several times** at the source until one tuple that satisfies the condition C is found



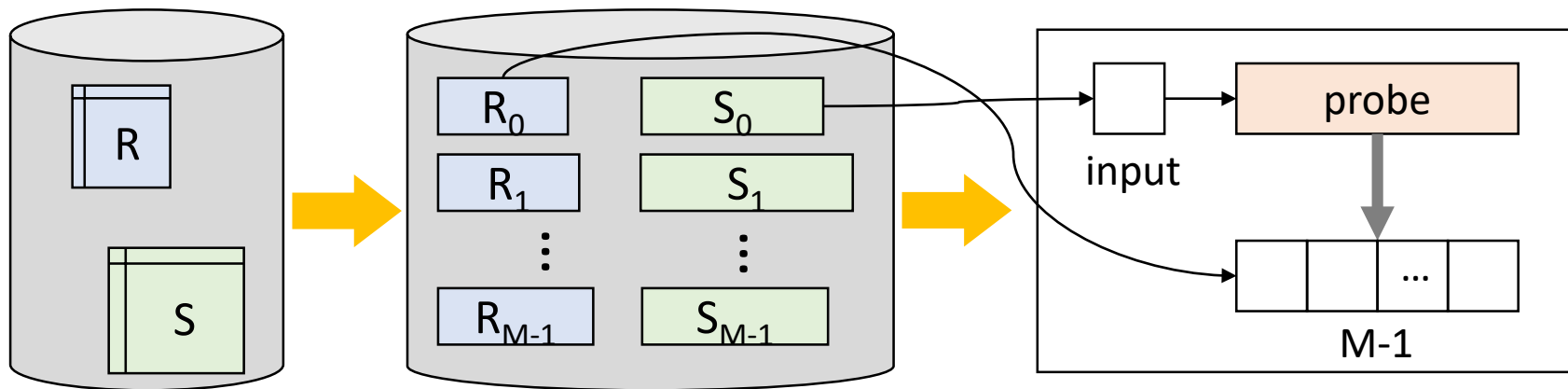
Pipelining binary operations

- Example: $(R(w, x) \bowtie S(x, y)) \bowtie U(y, z)$
 - hash-join is used for both join operations
 - either one-pass or two-pass depending on k
 - $M = 101$
- Two-pass hash joins (neither relation fits in main memory)
 - pass 1: R and S are partitioned into 100 buckets
 - R's bucket: approximately 50 blocks, S's bucket: approximately 100 blocks
 - pass 2: use 51 buffers (1 buffer for input)



Hash-based join (**hash-join**) (remind)

- Pass 1: partition both R and S to $M-1$ buckets
 - hash key: join attribute
- Pass 2: perform hash-join on each pair of buckets
 - **build** a hash table for **a smaller chunk**
 - **probe** the tuples of **a larger chunk** against the hash table

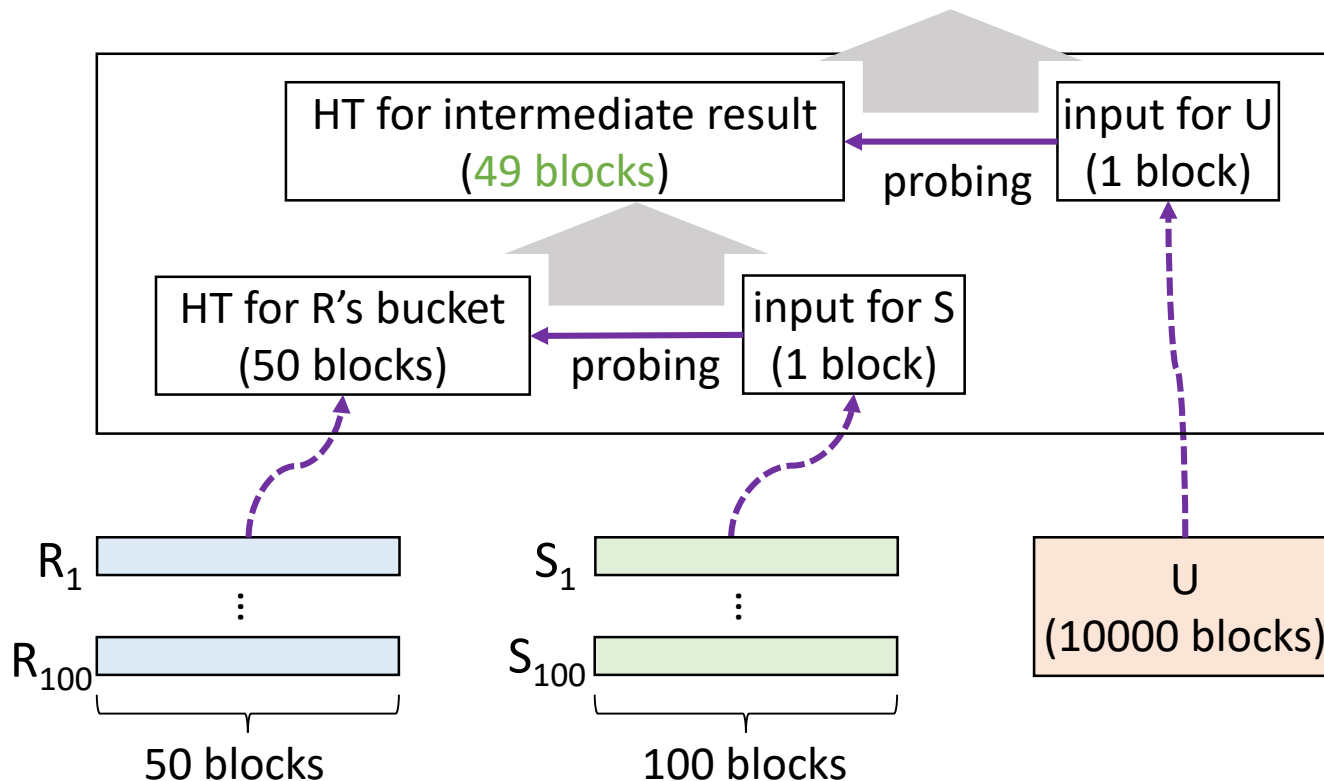


$B(R) / (M-1)$
on average

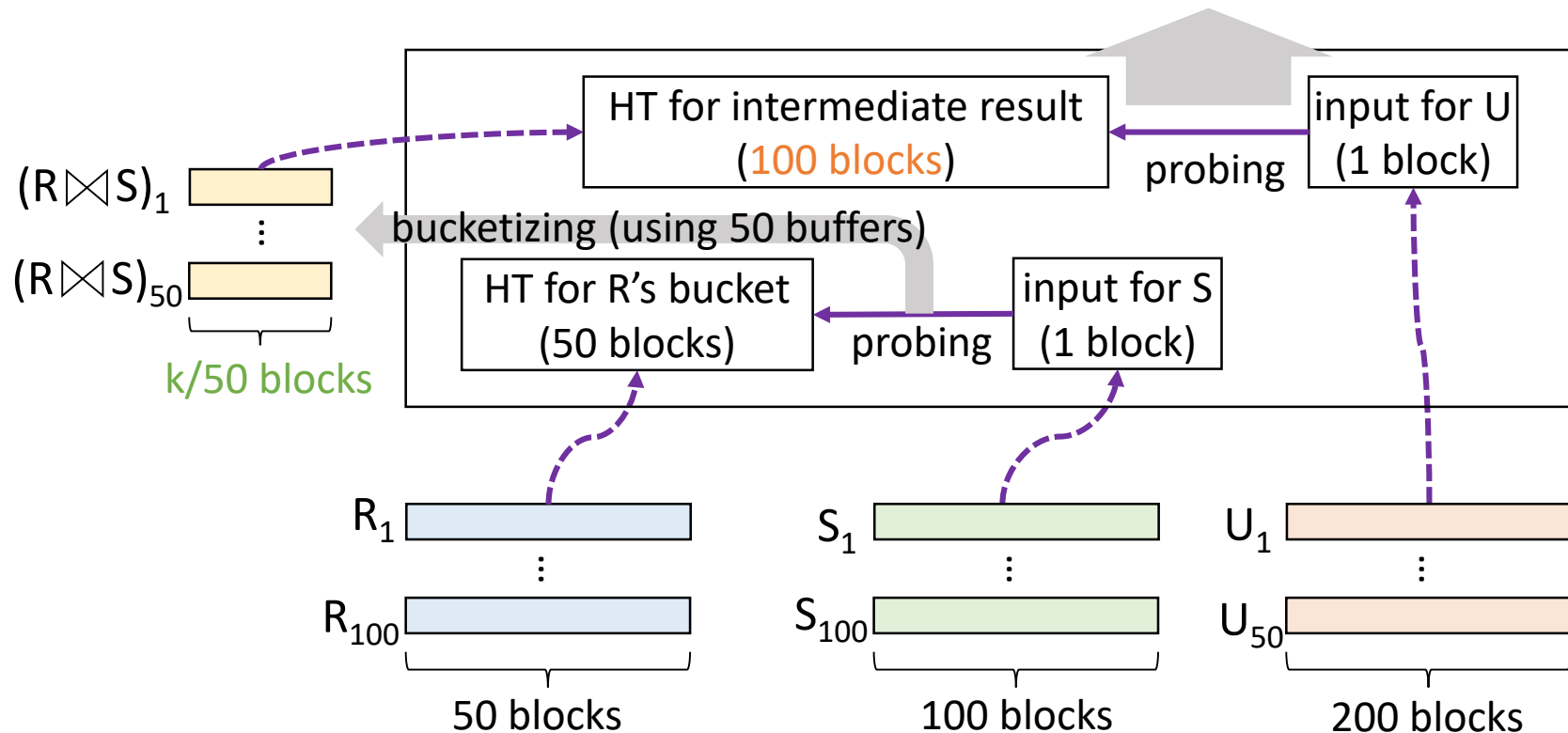
the second pass of the first join uses **51** buffers
the second join can use $101 - 51 = 50$ buffers

- Case 1: $k \leq 49$ (pipelining)

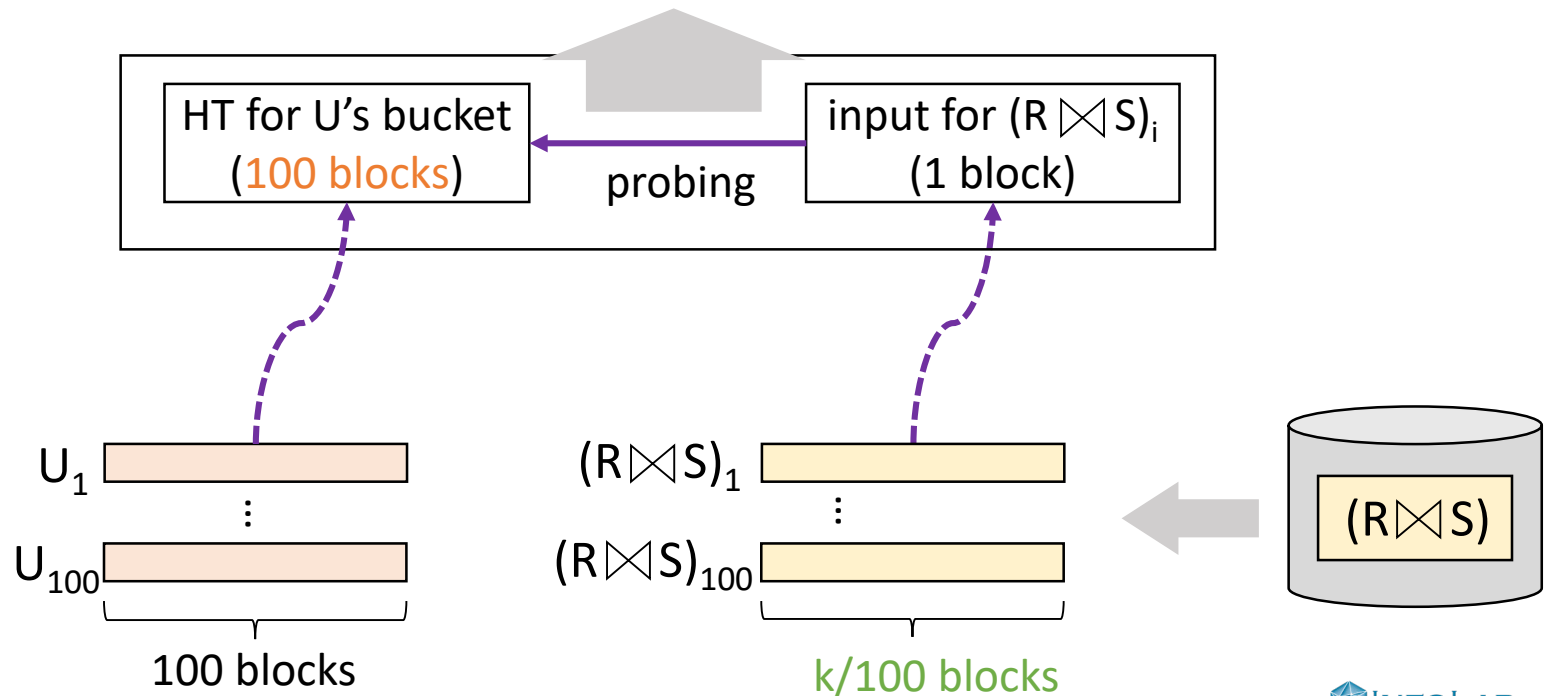
- result of $R \bowtie S$ occupies at most 49 blocks
- build a hash table for the intermediate result of $R \bowtie S$
- cost of the 1st join (two-pass): $15,000 + 30,000 = 45,000$ I/O's
- cost of the 2nd join (one-pass): $10,000$ I/O's
- total $55,000$ I/O's



- Case 2: $49 < k \leq 5000$ (materialization)
 - cost of the 1st join (two-pass): $15,000 + 30,000 = 45,000$ I/O's
 - writing the intermediate result: k I/O's
 - cost of the 2nd join (two-pass): $k + 20,000 + 10,000$ I/O's
 - total $75,000 + 2k$ I/O's



- Case 3: $5000 < k$ (materialization)
 - cost of the 1st join (two-pass): $15,000 + 30,000 = 45,000$ I/O's
 - writing the intermediate result: k I/O's
 - cost of the 2nd join (two-pass): $30,000 + 3k$ I/O's
 - total $75,000 + 4k$ I/O's

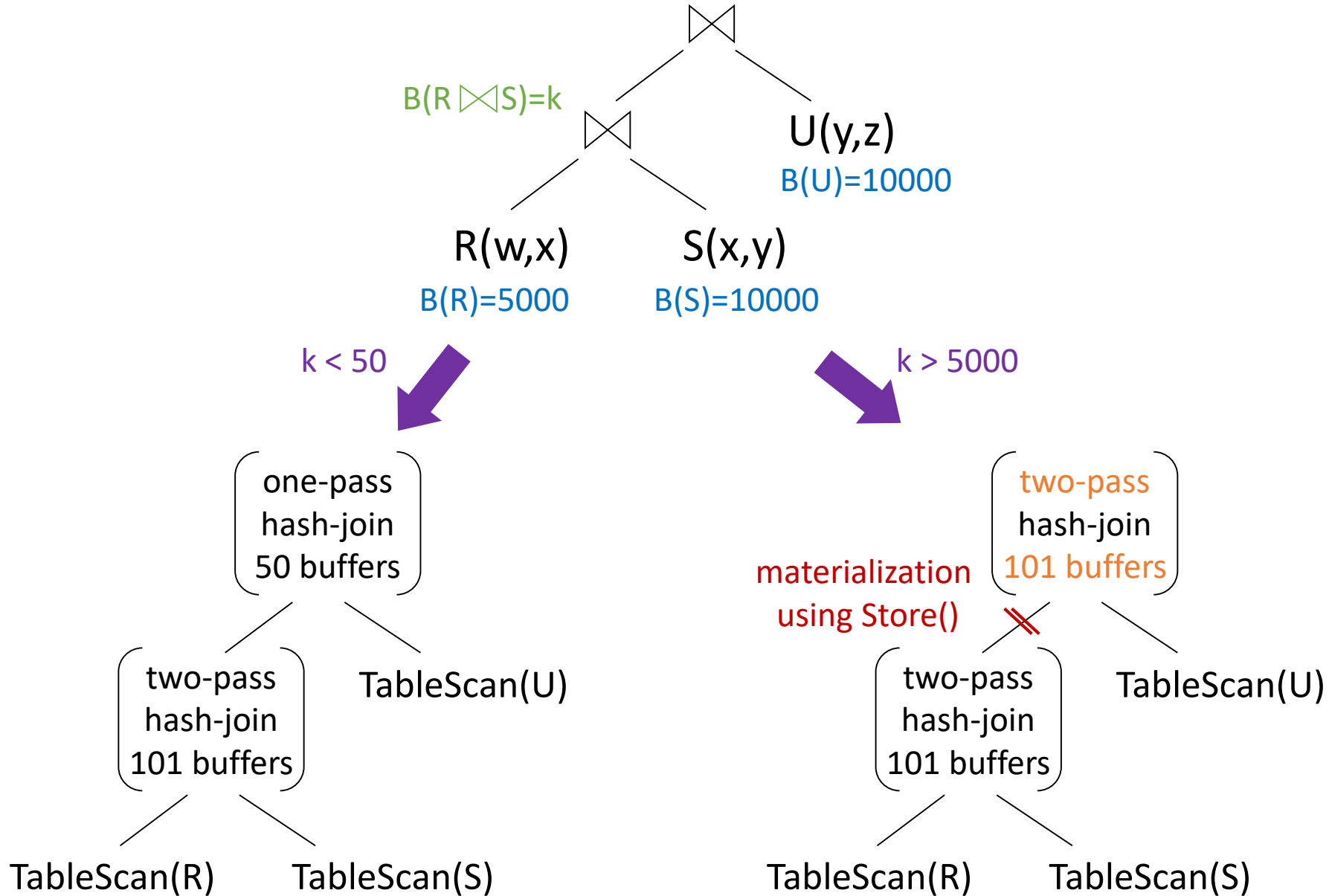


Notation for physical query plans

- Each DBMS uses its own internal notation
 - unlike the relational algebra, whose notation is fairly standard
- Physical operators for **leaf nodes** (of a physical query plan)
 - **TableScan(R)**: read R in arbitrary order
 - **SortScan(R, L)**: read R in order, sorted according to the list L
 - **IndexScan(R, C)**: access the tuples of R satisfying C through an index
 - C is a condition of the form $A \theta c$
 - index must be the one that supports range queries if θ is not =
 - **IndexScan(R, A)**: retrieve the tuple of R via an index on R.A
 - behave like TableScan, but may be more efficient if R is not clustered

- Physical operators for **selection** $\sigma_C(R)$
 - combined with the scan operators for a stored relation R
 - e.g., $\text{SortScan}(R, L) + \text{filter}(C)$, or $\text{TableScan}(R) + \text{filter}(C)$
 - if $C = A\theta c$ AND D , and there is an index on R.A,
then, $\text{IndexScan}(R, A\theta c) + \text{filter}(D)$
- Physical operator for **sort**
 - $\text{Sort}(L)$: explicit physical operator (e.g., ORDER BY clause)
- Physical operators for **other relational-algebra operations**
 - **operation** (e.g., join)
 - **parameters** (e.g., condition of a theta-join, grouping attributes)
 - **strategy** (e.g., sort-based, hash-based, index-based)
 - **# of passes** (e.g., one-pass, two-pass, multi-pass)
 - **estimated # of memory buffers** that the operation will require

Example of notation for the query in p.72



Thank you

Q & A